

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 13 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMMA)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define WARP_SIZE 32
129 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
130 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
131 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
132
133 // =====
134 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
135 // =====
136
137 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
138 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
139 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
140 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
141 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
142 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
143 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
144 //
145 // 初始: 每个 lane 持有自己的值 v0..v7
146 // lane:  0    1    2    3    4    5    6    7
147 // val:  v0   v1   v2   v3   v4   v5   v6   v7
148 //
149 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
150 //
151 //      ┌──────────┐
152 // 对:   (0,4) (1,5) (2,6) (3,7)
153 //
154 // lane:  0    1    2    3    4    5    6    7
155 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
156 //
157 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
158 //
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:   (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ───      ─── 后4个一组 ───
162 //
163 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
164 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //
169 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
170 // 对:   (0,1) (2,3) (4,5) (6,7)
171 //
172 // lane:  0    1    2    3    4    5    6    7
173 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
174 //
175 // mask=0: 循环终止, 归约完成。
176 //
177 // 关键性质:
178 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
179 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
180 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
181 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
182 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
183 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
184 template <const int kWarpSize = WARP_SIZE, typename T = float>
185 __device__ __forceinline__ T warp_reduce_sum(T val) {
186 #pragma unroll
187   for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
188     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
189   }
190   return val;
191 }

```

```

189
190 // Warp Reduce Max — generic
191 template <const int kWarpSize = WARP_SIZE, typename T = float>
192 __device__ __forceinline__ T warp_reduce_max(T val) {
193 #pragma unroll
194     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
195         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
196     }
197     return val;
198 }
199
200 // =====
201 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
202 // =====
203
204 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
205 // 两级归约流程:
206 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
207 // 2. warp leader (lane=0) 写入 shared memory
208 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
209 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
210 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
211 template <const int NUM_THREADS = 256>
212 __device__ float block_reduce_sum(float val) {
213     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
214     int warp = threadIdx.x / WARP_SIZE;
215     int lane = threadIdx.x % WARP_SIZE;
216     __shared__ float shared[NUM_WARPS];
217
218     float value = warp_reduce_sum<WARP_SIZE>(val);
219     if (lane == 0)
220         shared[warp] = value;
221     __syncthreads();
222     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
223     value = warp_reduce_sum<NUM_WARPS>(value);
224     // 关键: broadcast 结果到所有线程, 后续用 result
225     // 做除法等操作时所有线程都能拿到
226     value = __shfl_sync(0xffffffff, value, 0, 32);
227     return value;
228 }
229
230 // Block Reduce Max — FP32 (增强版, 带 broadcast)
231 template <const int NUM_THREADS = 256>
232 __device__ float block_reduce_max(float val) {
233     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
234     int warp = threadIdx.x / WARP_SIZE;
235     int lane = threadIdx.x % WARP_SIZE;
236     __shared__ float shared[NUM_WARPS];
237
238     float value = warp_reduce_max<WARP_SIZE>(val);
239     if (lane == 0)
240         shared[warp] = value;
241     __syncthreads();
242     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
243     value = warp_reduce_max<NUM_WARPS>(value);
244     value = __shfl_sync(0xffffffff, value, 0, 32);
245     return value;
246 }
247
248 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
249 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
250 // 跨 block 求和的常见模式, 适合 N 较大时使用;
251 // Grid: ((N + 255) / 256, 1, 1)

```

```

252 // Block: (256, 1, 1)
253 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
254 template <const int NUM_THREADS = 256>
255 __global__ void block_reduce_all(float *a, float *y, int N) {
256     int tid = threadIdx.x;
257     int idx = blockIdx.x * NUM_THREADS + tid;
258     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
259     __shared__ float shared[NUM_WARPS];
260
261     float val = (idx < N) ? a[idx] : 0.0f;
262     int warp = tid / WARP_SIZE;
263     int lane = tid % WARP_SIZE;
264
265     val = warp_reduce_sum<WARP_SIZE>(val);
266     if (lane == 0)
267         shared[warp] = val;
268     __syncthreads();
269
270     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
271     if (warp == 0)
272         val = warp_reduce_sum<NUM_WARPS>(val);
273     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
274         atomicAdd(y, val);
275 }
276
277 // ---- Dot Product: y = sum(a[i] * b[i]) ----
278 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
279 // Grid: ((N + 255) / 256, 1, 1)
280 // Block: (256, 1, 1)
281 // source: LeetCUDA/kernels/dot-product/dot_product.cu
282 template <const int NUM_THREADS = 256>
283 __global__ void dot(float *a, float *b, float *y, int N) {
284     int tid = threadIdx.x;
285     int idx = blockIdx.x * NUM_THREADS + tid;
286     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
287     __shared__ float shared[NUM_WARPS];
288
289     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
290     int warp = tid / WARP_SIZE;
291     int lane = tid % WARP_SIZE;
292
293     prod = warp_reduce_sum<WARP_SIZE>(prod);
294     if (lane == 0)
295         shared[warp] = prod;
296     __syncthreads();
297
298     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
299     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
300         prod = warp_reduce_sum<NUM_WARPS>(prod);
301     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
302         atomicAdd(y, prod);
303 }
304
305 // Dot Product + float4
306 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
307 // Block: (64, 1, 1), 256/4=64
308 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
309 // source: LeetCUDA/kernels/dot-product/dot_product.cu
310 template <const int NUM_THREADS = 256 / 4>
311 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
312     int tid = threadIdx.x;
313     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
314     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;

```

```

315 __shared__ float shared[NUM_WARPS];
316
317 float4 reg_a = FLOAT4(a[idx]);
318 float4 reg_b = FLOAT4(b[idx]);
319 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
320                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
321                          : 0.0f;
322 int warp = tid / WARP_SIZE;
323 int lane = tid % WARP_SIZE;
324
325 prod = warp_reduce_sum<WARP_SIZE>(prod);
326 if (lane == 0)
327     shared[warp] = prod;
328 __syncthreads();
329
330 prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
331 if (warp == 0)
332     prod = warp_reduce_sum<NUM_WARPS>(prod);
333 if (tid == 0)
334     atomicAdd(y, prod);
335 }
336
337
338 // =====
339 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
340 // =====
341 // 面试要点:
342 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
343 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
344 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
345
346 // ---- ReLU: y = max(0, x) ----
347 // Grid: ((N + 255) / 256, 1, 1)
348 // Block: (256, 1, 1)
349 // source: LeetCUDA/kernels/relu/relu.cu
350 __global__ void relu(float *x, float *y, int N) {
351     int idx = blockIdx.x * blockDim.x + threadIdx.x;
352     if (idx < N)
353         y[idx] = fmaxf(0.0f, x[idx]);
354 }
355
356 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
357 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
358 // Grid: ((N + 255) / 256, 1, 1)
359 // Block: (64, 1, 1)
360 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
361 // source: LeetCUDA/kernels/relu/relu.cu
362 __global__ void relu_vec4(float *x, float *y, int N) {
363     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
364     if (idx < N) {
365         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
366         float4 reg_y;
367         reg_y.x = fmaxf(0.0f, reg_x.x);
368         reg_y.y = fmaxf(0.0f, reg_x.y);
369         reg_y.z = fmaxf(0.0f, reg_x.z);
370         reg_y.w = fmaxf(0.0f, reg_x.w);
371         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
372     }
373 }
374
375 // ---- Elementwise Add: c = a + b ----
376 // Grid: ((N + 255) / 256, 1, 1)
377 // Block: (256, 1, 1)

```

```

378 // source: LeetCUDA/kernels/elementwise/elementwise.cu
379 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
380     int idx = blockIdx.x * blockDim.x + threadIdx.x;
381     if (idx < N)
382         c[idx] = a[idx] + b[idx];
383 }
384
385 // Elementwise Add + float4 向量化
386 // Grid: ((N + 255) / 256, 1, 1)
387 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
388 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
389 // source: LeetCUDA/kernels/elementwise/elementwise.cu
390 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
391     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
392     if ((idx + 3) < N) {
393         float4 reg_a = FLOAT4(a[idx]);
394         float4 reg_b = FLOAT4(b[idx]);
395         float4 reg_c;
396         reg_c.x = reg_a.x + reg_b.x;
397         reg_c.y = reg_a.y + reg_b.y;
398         reg_c.z = reg_a.z + reg_b.z;
399         reg_c.w = reg_a.w + reg_b.w;
400         FLOAT4(c[idx]) = reg_c;
401     } else if (idx < N) {
402         for (int i = 0; (idx + i) < N; i++) {
403             c[idx + i] = a[idx + i] + b[idx + i];
404         }
405     }
406 }
407
408 // ---- Histogram: y[a[i]]++ ----
409 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
410 // Grid: ((N + 255) / 256, 1, 1)
411 // Block: (256, 1, 1)
412 // source: LeetCUDA/kernels/histogram/histogram.cu
413 __global__ void histogram(int *a, int *y, int N) {
414     int idx = blockIdx.x * blockDim.x + threadIdx.x;
415     if (idx < N)
416         atomicAdd(&(y[a[idx]]), 1);
417 }
418
419 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
420 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
421 //
422 // 面试要点:
423 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
424 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
425 // - 数学公式 (5 步推导):
426 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
427 //       L_max = max(LSE_1, LSE_2)
428 //   Step 2 — 还原指数权重 (un-normalized):
429 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
430 //   Step 3 — 归一化为混合比例:
431 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
432 //       满足 alpha + beta = 1
433 //   Step 4 — 逐元素加权合并:
434 //       O = alpha * O_1 + beta * O_2
435 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
436 //   lse[head_idx][token_idx])
437 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
438 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
439 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score
440 //   为 -inf) 的 LSE 可能为 +inf, 替换后 exp(-inf - L_max) = 0,

```



```

441 // 该段权重退化为 0
442 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
443 //   pack load → FMA → pack store 一条流水线
444 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
445 //
446 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
447 //   pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
448 //   threads_per_head = head_size / 4 = 2
449 //   total_threads = num_tokens * num_heads * threads_per_head = 8
450 //
451 //   global_idx:      0      1      2      3      4      5      6      7
452 //   token_head_idx:  0      0      1      1      2      2      3      3
453 //   (第几个 (token,head) 对, 行优先)
454 //   pack_idx:        0      1      0      1      0      1      0      1
455 //   (该 (token,head) 对内第几个 pack)
456 //   token_idx:        0      0      0      0      1      1      1      1
457 //   (token_head_idx / num_heads, token 变化慢)
458 //   head_idx:         0      0      1      1      0      0      1      1
459 //   (token_head_idx % num_heads, head 变化快)
460 //   pack_offset:      0      4      0      4      0      4      0      4
461 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
462 //   head_offset:      0      0      8      8      16     16     24     24
463 //   (token_idx * num_heads * head_size + head_idx * head_size,
464 //    该 (token,head) 对在 output 展平数组中的起始偏移)
465 //
466 // Grid: ((total_threads + 127) / 128, 1, 1)
467 // Block: (128, 1, 1)
468 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
469 _global__ void merge_attn_states(
470     float *output,           // [num_tokens, num_heads, head_size]
471     const float *prefix_output, // [num_tokens, num_heads, head_size]
472     const float *prefix_lse,   // [num_heads, num_tokens]
473     const float *suffix_output, // [num_tokens, num_heads, head_size]
474     const float *suffix_lse,   // [num_heads, num_tokens]
475     int num_tokens, int num_heads, int head_size) {
476
477     constexpr int NUM_THREADS = 128;
478     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
479     using pack_t = uint4;
480
481     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
482     const int threads_per_head = head_size / PACK_SIZE;
483     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
484     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
485     const int total_threads = num_tokens * num_heads * threads_per_head;
486
487     const int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
488     if (global_idx >= total_threads)
489         return;
490
491     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
492     // token_head_idx: 第几个 (token, head) 对, 行优先展平
493     const int token_head_idx = global_idx / threads_per_head;
494     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
495     const int pack_idx = global_idx % threads_per_head;
496
497     // token_head_idx 分解为 (token_idx, head_idx)
498     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
499     const int head_idx = token_head_idx % num_heads;   // head 变化快 (内维)
500
501     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
502     const int pack_offset = pack_idx * PACK_SIZE;
503     // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移

```

```

504 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
505
506 // 定位到当前 (token, head) 的输出段起始
507 const float *prefix_head = prefix_output + head_offset;
508 const float *suffix_head = suffix_output + head_offset;
509 float *output_head = output + head_offset;
510
511 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
512 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
513 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
514
515 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
516 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
517 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
518
519 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
520 const float max_lse = fmaxf(p_lse, s_lse);
521 p_lse -= max_lse;
522 s_lse -= max_lse;
523
524 // Step 2-3: 指数还原 → 混合比例 alpha, beta
525 const float p_se = expf(p_lse);
526 const float s_se = expf(s_lse);
527 const float out_se = p_se + s_se;
528 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
529 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
530
531 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
532 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
533 if (pack_offset < head_size) {
534     pack_t p_pack =
535         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
536     pack_t s_pack =
537         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
538     pack_t o_pack;
539
540 #pragma unroll
541     for (int i = 0; i < PACK_SIZE; ++i) {
542         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
543         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
544         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
545         reinterpret_cast<float *>(&o_pack)[i] = o_v;
546     }
547
548     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
549 }
550 }
551
552 // =====
553 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
554 // =====
555 // 面试要点:
556 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
557 //     online (1-pass, 增量更新)
558 //   - Online Softmax 是 FlashAttention 的数学基础
559 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
560 //     + variance)
561 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
562
563 // =====
564 // Phase 3a: Softmax — 三级递进 (面试核心考点)
565 // =====
566 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」

```

```

567 // 回答线索: naive(溢出) → safe → online
568
569 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
570 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
571 // 问题: x 值很大时 exp(x) 溢出为 inf
572 template <const int NUM_THREADS = 256>
573 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
574 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
575 // source: LeetCUDA/kernels/softmax/softmax.cu
576 __global__ void softmax_per_token(float *x, float *y, int N) {
577     const int tid = threadIdx.x;
578     const int idx = blockIdx.x * blockDim.x + tid;
579
580     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
581     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
582     if (idx < N)
583         y[idx] = exp_val / exp_sum;
584 }
585
586 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
587 // 面试重点: 为什么 Softmax 需要 Safe?
588 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
589 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
590 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
591 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
592 // Grid: (S, 1, 1)
593 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
594 // source: LeetCUDA/kernels/softmax/softmax.cu
595 template <const int NUM_THREADS = 256>
596 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
597     const int tid = threadIdx.x;
598     const int idx = blockIdx.x * blockDim.x + tid;
599
600     // Pass 1: block reduce max — 找最大值
601     float val = (idx < N) ? x[idx] : -FLT_MAX;
602     float max_val = block_reduce_max<NUM_THREADS>(val);
603
604     // Pass 2: exp(x - max) → block reduce sum
605     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
606     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
607
608     if (idx < N)
609         y[idx] = exp_val / exp_sum;
610 }
611
612 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
613 // 面试重点:
614 // 两种使用场景 (公式不同, 但结果等价):
615 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
616 //     m_new = max(m_old, x_i)
617 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
618 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
619 //     m = max(m1, m2) safe softmax用的max值
620 //     d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
621 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
622 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
623 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
624 //
625 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
626 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
627 // (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
628 // (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2
629 //

```

```

630 // 合并公式 (对称, 不依赖"新/旧"概念):
631 //   m = max(m1, m2)
632 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
633 //
634 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
635 struct __align__(8) MD {
636     float m; // running max
637     float d; // running denominator (sum of exp(x - max))
638 };
639
640 template <const int kWarpSize = WARP_SIZE>
641 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
642     #pragma unroll
643     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
644         MD md2;
645         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
646         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
647
648         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
649         MD s_m = (md1.m > md2.m) ? md2 : md1;
650
651         // b_m.d 无需 rescale (其基准 max 已是全局 max),
652         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
653         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
654         md1.m = b_m.m;
655     }
656     return md1;
657 }
658
659 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
660 // Grid: (S, 1, 1)
661 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
662 // source: LeetCUDA/kernels/softmax/softmax.cu
663 template <const int NUM_THREADS = 256>
664 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
665     int tid = threadIdx.x;
666     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
667     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
668     __shared__ MD shared[NUM_WARPS];
669
670     float val = (idx < N) ? x[idx] : -FLT_MAX;
671     int warp = tid / WARP_SIZE;
672     int lane = tid % WARP_SIZE;
673
674     // 初始化: 每个线程持有一个 (max, denom) 对
675     MD md;
676     md.m = idx < N ? val : -FLT_MAX;
677     md.d = idx < N ? 1.0f : 0.0f;
678
679     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
680     md = warp_reduce_md<WARP_SIZE>(md);
681
682     if (lane == 0)
683         shared[warp] = md;
684     __syncthreads();
685
686     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
687     md = lane < NUM_WARPS ? shared[lane] : MD{-FLT_MAX, 0.0f};
688     md = warp_reduce_md<WARP_SIZE>(md); // 用WARP_SIZE确保每个lane都能拿到最终结果
689
690     // 用全局 max 和 denom 做最终 softmax
691     float d_inv = __fdividef(1.0f, md.d);
692     // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径

```

```

693     if (idx < N) {
694         y[idx] = __expf(val - md.m) * d_inv;
695     }
696 }
697
698 // =====
699 // Phase 3b: RMS Normalization (1-pass reduce)
700 // =====
701 // 面试要点:
702 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
703 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
704 //   - Llama 系列使用 RMS Norm
705 //   - grid(N, K/K), block(K): 一行一个 block
706 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
707 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
708 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
709 template <const int NUM_THREADS = 128>
710 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
711     int tid = threadIdx.x;
712     int idx = blockIdx.x * blockDim.x + threadIdx.x;
713     const float epsilon = 1e-5f;
714
715     __shared__ float s_variance;
716     float value = (idx < N * K) ? x[idx] : 0.0f;
717     float variance = value * value;
718     variance = block_reduce_sum<NUM_THREADS>(variance);
719     if (tid == 0)
720         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
721     __syncthreads();
722     if (idx < N * K)
723         y[idx] = (value * s_variance) * g;
724 }
725
726 // RMS Norm + float4
727 // Grid: (N, 1, 1)
728 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
729 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
730 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
731 template <const int NUM_THREADS = 128 / 4>
732 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
733     int tid = threadIdx.x;
734     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
735     const float epsilon = 1e-5f;
736
737     __shared__ float s_variance;
738     float4 reg_x = FLOAT4(x[idx]);
739     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
740                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
741                         : 0.0f;
742     variance = block_reduce_sum<NUM_THREADS>(variance);
743     if (tid == 0)
744         s_variance = rsqrtf(variance / (float)K + epsilon);
745     __syncthreads();
746     float4 reg_y;
747     reg_y.x = reg_x.x * s_variance * g;
748     reg_y.y = reg_x.y * s_variance * g;
749     reg_y.z = reg_x.z * s_variance * g;
750     reg_y.w = reg_x.w * s_variance * g;
751     if (idx < N * K)
752         FLOAT4(y[idx]) = reg_y;
753 }
754
755 // =====

```

```

756 // Phase 3c: Layer Normalization (2-pass reduce)
757 // =====
758 // 面试要点:
759 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
760 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
761 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
762 // Grid: (N, 1, 1), 一行一个 block
763 // Block: (128, 1, 1), NUM_THREADS=K=128
764 // source: LeetCode/kernels/layer-norm/layer_norm.cu
765 template <const int NUM_THREADS = 128>
766 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
767     int tid = threadIdx.x;
768     int idx = blockIdx.x * blockDim.x + threadIdx.x;
769     const float epsilon = 1e-5f;
770
771     __shared__ float s_mean;
772     __shared__ float s_variance;
773     float value = (idx < N * K) ? x[idx] : 0.0f;
774
775     // Pass 1: compute mean
776     float sum = block_reduce_sum<NUM_THREADS>(value);
777     if (tid == 0)
778         s_mean = sum / (float)K;
779     __syncthreads(); // 必须等待 s_mean 对所有线程可见
780
781     // Pass 2: compute variance = (x - mean)2
782     float variance = (value - s_mean) * (value - s_mean);
783     variance = block_reduce_sum<NUM_THREADS>(variance);
784     if (tid == 0)
785         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
786     __syncthreads(); // 必须等待 s_variance 对所有线程可见
787
788     if (idx < N * K)
789         y[idx] = ((value - s_mean) * s_variance) * g + b;
790 }
791
792 // Layer Norm + float4
793 // Grid: (N, 1, 1)
794 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
795 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
796 // source: LeetCode/kernels/layer-norm/layer_norm.cu
797 template <const int NUM_THREADS = 128 / 4>
798 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
799     int tid = threadIdx.x;
800     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
801     const float epsilon = 1e-5f;
802
803     __shared__ float s_mean;
804     __shared__ float s_variance;
805     float4 reg_x = FLOAT4(x[idx]);
806     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
807
808     float sum = block_reduce_sum<NUM_THREADS>(value);
809     if (tid == 0)
810         s_mean = sum / (float)K;
811     __syncthreads();
812
813     float4 reg_x_hat;
814     reg_x_hat.x = reg_x.x - s_mean;
815     reg_x_hat.y = reg_x.y - s_mean;
816     reg_x_hat.z = reg_x.z - s_mean;
817     reg_x_hat.w = reg_x.w - s_mean;
818     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +

```

```

819         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
820     variance = block_reduce_sum<NUM_THREADS>(variance);
821     if (tid == 0)
822         s_variance = rsqrtf(variance / (float)K + epsilon);
823     __syncthreads();
824
825     float4 reg_y;
826     reg_y.x = reg_x_hat.x * s_variance * g + b;
827     reg_y.y = reg_x_hat.y * s_variance * g + b;
828     reg_y.z = reg_x_hat.z * s_variance * g + b;
829     reg_y.w = reg_x_hat.w * s_variance * g + b;
830     if (idx < N * K)
831         FLOAT4(y[idx]) = reg_y;
832 }
833
834 // =====
835 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
836 // =====
837 // 面试要点:
838 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
839 //       [x1']   [cos(θ)  -sin(θ)] [x1]
840 //       [x2'] = [sin(θ)   cos(θ)] [x2]
841 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
842 //   - token_pos = idx / N: token 在序列中的位置
843 //   - token_idx = idx % N: token 内的维度索引
844 //   - 输入 [seq_len, hidden_size], 输出同形状
845
846 // Grid: ((seq_len * N + 255) / 256, 1, 1)
847 // Block: (256, 1, 1)
848 // source: LeetCUDA/kernels/rope/rope.cu
849 __global__ void rope(float *x, float *out, int seq_len, int N) {
850     int idx = blockIdx.x * blockDim.x + threadIdx.x;
851     float x1 = x[idx * 2];
852     float x2 = x[idx * 2 + 1];
853     int token_pos = idx / N; // 序列位置
854     int token_idx = idx % N; // 维度索引
855
856     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
857     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
858     float sin_v = sinf(token_pos * exp_v);
859     float cos_v = cosf(token_pos * exp_v);
860
861     // 2D 旋转
862     float out1 = x1 * cos_v - x2 * sin_v;
863     float out2 = x1 * sin_v + x2 * cos_v;
864     out[idx * 2] = out1;
865     out[idx * 2 + 1] = out2;
866 }
867
868 // =====
869 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
870 // =====
871 // 面试要点 (Bank Conflict 专题):
872 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
873 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
874 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
875 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
876 //
877 // 转置的四步演进:
878 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
879 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
880 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
881 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store

```



```

882 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
883
884 // 每个线程处理 1 个元素, block(16,16)
885 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
886 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
887 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
888 // Block: (16, 16, 1)
889 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
890 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
891     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
892     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
893     if (r < row && c < col) {
894         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
895         y[c * row + r] = x[r * col + c];
896     }
897 }
898
899 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
900 // Block: (16, 16, 1)
901 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose_padded(
904     float *x, float *y, const int row, const int col) {
905     const int tx = threadIdx.x;
906     const int ty = threadIdx.y;
907
908     constexpr int TILE = 16;
909     constexpr int PAD = 1;
910     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
911     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
912
913     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
914     const int x_c = blockIdx.x * TILE + tx;
915     const int x_r = blockIdx.y * TILE + ty;
916
917     if (x_r * 4 < row && x_c < col) {
918         // Step 1: 4 rows × 1 col → smem (row-major);
919 #pragma unroll
920         for (int i = 0; i < 4; ++i) {
921             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
922         }
923         __syncthreads();
924
925         // Step 2: Transposed read from smem
926         float4 reg_trans;
927         reg_trans.x = tile[tx * 4 + 0][ty];
928         reg_trans.y = tile[tx * 4 + 1][ty];
929         reg_trans.z = tile[tx * 4 + 2][ty];
930         reg_trans.w = tile[tx * 4 + 3][ty];
931
932         // y 空间坐标: y_r = x 的列, y_c = x 的行
933         const int y_r = blockIdx.x * TILE + ty; // = x col
934         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
935
936         // Coalesced write: y[y_r][y_c .. y_c+3]
937         FLOAT4(y[y_r * row + y_c]) = reg_trans;
938     }
939 }
940
941 // =====
942 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
943 // =====
944 // 面试要点:

```



```

945 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
946 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
947 // - 不同 K 值对应不同分块策略:
948 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
949 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
950 //   K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
951 //
952 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
953
954 // ---- SGEMV K32: 基础 warp-per-row ----
955 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
956 // grid(M/4), 每个 warp 负责一行
957 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
958 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
959 // Block: (32, 4, 1), 每行 1 warp
960 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
961 // source: LeetCUDA/kernels/sgemv/sgemv.cu
962 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
963     int tx = threadIdx.x; // 0~31
964     int ty = threadIdx.y; // 0~3
965     int lane = tx % WARP_SIZE; // 0~31
966     int m = blockIdx.x * blockDim.y + ty; // 全局行号
967     if (m < M) {
968         float sum = 0.0f;
969         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
970         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
971         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
972 #pragma unroll
973         for (int w = 0; w < NUM_ITERS; ++w) {
974             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
975             int k = w * WARP_SIZE + lane;
976             sum += a[m * K + k] * x[k];
977         }
978         sum = warp_reduce_sum<WARP_SIZE>(sum);
979         // 每个 warp 处理一行, lane 0 写回结果
980         if (lane == 0)
981             y[m] = sum;
982     }
983 }
984
985 // ---- SGEMV K128: float4 向量化 ----
986 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
987 // Grid: ((M + 3) / 4, 1, 1)
988 // Block: (32, 4, 1)
989 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
990 // source: LeetCUDA/kernels/sgemv/sgemv.cu
991 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
992     int tx = threadIdx.x; // 0~31
993     int ty = threadIdx.y; // 0~3
994     int lane = tx % WARP_SIZE; // 0~31
995     int m = blockDim.y * blockIdx.x + ty;
996
997     if (m < M) {
998         float sum = 0.0f;
999         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1000         const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1001 #pragma unroll
1002         for (int w = 0; w < NUM_ITERS; ++w) {
1003             int k = (w * WARP_SIZE + lane) * 4;
1004             float4 reg_x = FLOAT4(x[k]);
1005             float4 reg_a = FLOAT4(a[m * K + k]);
1006             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1007                    reg_a.w * reg_x.w);

```

```

1008     }
1009     sum = warp_reduce_sum<WARP_SIZE>(sum);
1010     if (lane == 0)
1011         y[m] = sum;
1012 }
1013 }
1014
1015 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1016 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1017 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1018 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1019 // Block: (32, 4, 1)
1020 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1021 // source: LeetCode/kernels/sgemv/sgemv.cu
1022 template <const int ROW_PER_WARP = 2>
1023 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1024     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1025     int tx = threadIdx.x;
1026     int ty = threadIdx.y;
1027     int lane = tx % WARP_SIZE;
1028     int k = lane % K_WARP_SIZE; // 0~15
1029     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1030     if (m < M) {
1031         float sum = A[m * K + k] * x[k];
1032         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1033         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1034         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1035         if (k == 0)
1036             y[m] = sum;
1037     }
1038 }
1039
1040 // =====
1041 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1042 // =====
1043 // 面试要点 (GEMM 优化五层金字塔):
1044 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1045 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1046 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1047 //   load/store 指令数 Level 4 — Tensor Core (MMA
1048 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1049 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1050 //
1051 // 计算密度递进:
1052 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1053 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1054 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1055 //
1056 // =====
1057 // Phase 7a: SGEMM (非 Tensor Core 路径)
1058 // =====
1059
1060 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1061 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1062 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1063 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1064 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1065 // Block: (32, 32, 1), 1024 线程
1066 // source: LeetCode/kernels/sgemm/sgemm.cu
1067 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1068     constexpr int BM = 32; // vec 版: 32×4 = 128
1069     constexpr int BN = 32; // vec 版: 32×4 = 128
1070     constexpr int BK = 32;

```

```

1071 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1072
1073 int bx = blockIdx.x;
1074 int by = blockIdx.y;
1075 int tx = threadIdx.x;
1076 int tid = threadIdx.y * blockDim.x + tx;
1077
1078 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1079 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1080 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1081 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1082 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1083 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1084 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1085
1086 float sum = 0.f; // 遍历完整的K, slice K;
1087 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1088     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1089     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1090     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1091     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1092     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1093     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1094     __syncthreads(); // 确保整个 smem tile 加载完毕
1095
1096 #pragma unroll
1097     for (int k = 0; k < BK; ++k) {
1098         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1099         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1100         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1101     }
1102     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1103 }
1104 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1105 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1106 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1107 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1108 }
1109
1110 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1111 // 在 Level 1 基础上引入两层优化:
1112 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1113 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1114 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1115 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1116 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1117 // 1024 x 16 = 16384 = 128 x 128 ✓
1118 //
1119 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1120 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1121 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1122 //
1123 // 加载映射 (每线程 4 个元素, float4):
1124 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1125 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1126 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1127 //
1128 // △ Bank Conflict 提示 (面试加分点):
1129 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1130 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1131 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1132 //
1133 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)

```

```

1134 // Block: (32, 32, 1), 1024 线程
1135 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1136 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1137 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1138     constexpr int BM = 128;
1139     constexpr int BN = 128;
1140     constexpr int BK = 32;
1141     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1142     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1143
1144     int bx = blockIdx.x;
1145     int by = blockIdx.y;
1146     int tx = threadIdx.x;
1147     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1148
1149     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1150     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1151     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1152     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1153     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1154     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1155
1156     int load_gmem_a_m = by * BM + load_smem_a_m;
1157     int load_gmem_b_n = bx * BN + load_smem_b_n;
1158
1159     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1160     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1161     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1162     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1163     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1164     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1165
1166     float sum[4][4] = {0.f};
1167     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1168         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1169         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1170         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1171         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1172         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1173         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1174         __syncthreads();
1175
1176     #pragma unroll
1177     for (int k = 0; k < BK; ++k) {
1178         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1179         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1180                             s_a[comp_smem_a_m_base + 1][k],
1181                             s_a[comp_smem_a_m_base + 2][k],
1182                             s_a[comp_smem_a_m_base + 3][k]};
1183         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1184                             s_b[k][comp_smem_b_n_base + 1],
1185                             s_b[k][comp_smem_b_n_base + 2],
1186                             s_b[k][comp_smem_b_n_base + 3]};
1187
1188     #pragma unroll
1189     for (int i = 0; i < 4; ++i) {
1190     #pragma unroll
1191         for (int j = 0; j < 4; ++j) {
1192             sum[i][j] += a_vals[i] * b_vals[j];
1193         }
1194     }
1195     __syncthreads();
1196 }

```

```

1197 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1198 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1199 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1200 #pragma unroll
1201 for (int i = 0; i < 4; ++i) {
1202     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1203     float4 reg_c;
1204     reg_c.x = sum[i][0];
1205     reg_c.y = sum[i][1];
1206     reg_c.z = sum[i][2];
1207     reg_c.w = sum[i][3];
1208     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1209 }
1210 }
1211 }
1212
1213 // =====
1214 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1215 // =====
1216 // 面试要点:
1217 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1218 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1219 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1220 // 寄存器)
1221 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1222 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1223 //
1224 // ★ TN 布局详解 (面试高频考点):
1225 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1226 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1227 // N = Normal (列优先, BLAS 原生格式)
1228 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1229 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1230 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1231 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1232 // 视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1233 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1234 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1235 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1236 //
1237 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1238 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1239 // N = col-major (列优先, BLAS native = normal)
1240 //
1241 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1242 // - A: row-major [M, K], B: row-major [K, N]
1243 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1244 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1245 //
1246 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1247 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1248 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1249 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1250 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1251 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1252 //
1253 // WGMMMA (Warp Group MMA, Hopper+):
1254 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1255 // - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1256 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1257 // threads) 做计算
1258 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1259

```

```

1260 // =====
1261 // Phase 7b-1: MMA PTX 宏定义
1262 // =====
1263
1264 // ---- gmem → smem: cp.async ----
1265 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1266 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1267 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1268 //   N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1269 // - wait_all: 等价于 commit_group + wait_group 0。
1270 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1271 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1272 #define CP_ASYNC_WAIT_GROUP(n) \
1273     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1274
1275 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1276 #define CP_ASYNC_CG(dst, src, bytes) \
1277     asm volatile( \
1278         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1279         "l"(src), "n"(bytes))
1280
1281 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1282 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1283 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1284 // aligned: 要求 128-bit 对齐, trans: 转置加载
1285 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1286     asm volatile( \
1287         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1288         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1289         : "r"(addr))
1290
1291 #define LDMATRIX_X2(R0, R1, addr) \
1292     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1293         : "=r"(R0), "=r"(R1) \
1294         : "r"(addr))
1295
1296 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1297 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1298 #define LDMATRIX_X2_T(R0, R1, addr) \
1299     asm volatile( \
1300         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1301         : "=r"(R0), "=r"(R1) \
1302         : "r"(addr))
1303
1304 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1305 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1306 // row.col: A 是 row-major, B 是 col-major
1307 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1308 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1309 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1310 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1311     asm volatile( \
1312         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1313         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1314         : "=r"(RD0), "=r"(RD1) \
1315         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1316         "r"(RC1))
1317
1318 // ---- MMA 辅助函数 ----
1319 // div_ceil: 整数除法向上取整
1320 #define HOST_DEVICE_INLINE __device__ __host__ inline
1321 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1322     return (a % b != 0) ? (a / b + 1) : (a / b);

```



```

1323 }
1324
1325 // =====
1326 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1327 // =====
1328 // 面试重点 — Tile Hierarchy:
1329 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1330 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1331 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1332 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1333 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1334 //
1335 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1336 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1337 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1338 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1339 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1340 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1341 //
1342 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1343 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1344 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1345 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1346 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1347 //
1348 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1349 // Block: (256, 1, 1), 8 warps
1350 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1351 // =====
1352 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1353           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1354           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1355           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1356 __global__ void __launch_bounds__(256)
1357   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1358   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1359   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1360   const int by = blockIdx.y;
1361   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1362   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1363   constexpr int BK = MMA_K; // 16
1364
1365   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1366   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1367   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1368   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1369   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1370   extern __shared__ half smem[];
1371   half *s_a = smem;
1372   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1373   constexpr int s_a_stage_offset = BM * BK; // 128*16
1374   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1375   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1376   const int warp_id = tid / WARP_SIZE; // 0~7
1377   const int lane_id = tid % WARP_SIZE; // 0~31
1378   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1379   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1380   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1381   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1382
1383   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1384   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1385   int load_smem_a_m = tid / 2; // 0~127

```

```

1386 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1387 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1388 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1389 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1390 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1391 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1392     return;
1393
1394 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1395 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1396 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1397 // MMA Tile对应到cutlass cude中的TiledMMA的概念, Value Tile对应到cutlass cude中的PermuteMNK的概念。
1398 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1399
1400 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1401 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1402 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1403
1404 // 预加载前 (K_STAGE-1) 个 stage
1405 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1406 #pragma unroll
1407 for (int k = 0; k < (K_STAGE - 1); ++k) {
1408     int load_gmem_a_k = k * BK + load_smem_a_k;
1409     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1410     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1411         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1412     );
1413     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1414
1415     int load_gmem_b_k = k * BK + load_smem_b_k;
1416     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1417     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1418     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1419         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1420     );
1421     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1422
1423     CP_ASYNC_COMMIT_GROUP();
1424 }
1425
1426 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1427 __syncthreads();
1428
1429 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1430 const int NUM_K_TILES = div_ceil(K, BK);
1431 #pragma unroll
1432 for (int k = 0; k < NUM_K_TILES; ++k) {
1433     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1434     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1435
1436     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用。因为在prefetch loop中已经
1437     // loaded (K_STAGE-1) 个 stage, 因此这里是从 tile (k+K_STAGE-1) 开始预加载
1438     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1439     // (row-major, 内维连续的是 K)
1440     if (k + K_STAGE - 1 < NUM_K_TILES) {
1441         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1442         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1443         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1444         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1445
1446         uint32_t load_smem_a_ptr =
1447             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1448                 load_smem_a_m * BK + load_smem_a_k) *

```



```

1449         sizeof(half));
1450 CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1451
1452 uint32_t load_smem_b_ptr =
1453     (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1454         load_smem_b_n * BK + load_smem_b_k) *
1455         sizeof(half));
1456 CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1457 CP_ASYNC.COMMIT_GROUP();
1458 }
1459
1460 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1461 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1462 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1463 uint32_t RA[VAL_TILE_M][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1464 uint32_t RB[VAL_TILE_N][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1465
1466 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1467 #pragma unroll
1468 for (int i = 0; i < VAL_TILE_M; ++i) {
1469     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1470     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1471     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1472     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1473     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1474     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1475     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1476     uint32_t lane_smem_a_ptr =
1477         (smem_a_base_ptr +
1478         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1479         sizeof(half));
1480     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1481 }
1482
1483 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1484 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1485 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1486 #pragma unroll
1487 for (int j = 0; j < VAL_TILE_N; ++j) {
1488     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1489     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1490     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1491     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1492     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1493     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1494     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1495     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1496     uint32_t lane_smem_b_ptr =
1497         (smem_b_base_ptr +
1498         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1499         sizeof(half));
1500     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1501 }
1502
1503 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1504 #pragma unroll
1505 for (int i = 0; i < VAL_TILE_M; ++i) {
1506     #pragma unroll
1507     for (int j = 0; j < VAL_TILE_N; ++j) {
1508         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1509             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1510             RB[j][0], RB[j][1], // B fragment
1511             RC[i][j][0], RC[i][j][1]);

```

```

1512 }
1513 }
1514
1515 // 自适应等待：流水线满载期用 K_STAGE-2，尾部排空用 0
1516 if (k + K_STAGE - 1 < NUM_K_TILES) {
1517     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1518 } else { // 对于尾部的 k，等待所有剩余的 cp.async 完成
1519     CP_ASYNC_WAIT_GROUP(0);
1520 }
1521 __syncthreads();
1522 }
1523
1524 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1525 {
1526     for (int i = 0; i < VAL_TILE_M; ++i) {
1527         // RC[VAL_TILE_M][VAL_TILE_N][2] 中 RC[...][...][2] 中保存了2个 uint32
1528         // 寄存器，每个uint32 寄存器表示两个临近的fp16值：{c0,c1}，然后RC[...][...][0]
1529         // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上（不同物理行，跨8x8）
1530         // 同一个位置上的元素，也就是，实际代表了2个不同的行的元素，因此要分开RC0，RC1；
1531         // RC0表示第一行，8个half，可以用4个uint32寄存器来装；同理，RC1表示第二行。
1532         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1533         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1534         // =====
1535         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1536         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1537         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1538         //
1539         //      cols: c0-1   c2-3   c4-5   c6-7
1540         //  r0:  T0{c0,c1}  T1{c2,c3} T2{c4,c5} T3{c6,c7}  |
1541         //  r1:  T4{c0,c1}  T5{c2,c3} T6{c4,c5} T7{c6,c7}  |
1542         //  r2:  T8{c0,c1}  T9{c2,c3} T10{c4,c5} T11{c6,c7} | RC[0]
1543         //  ...
1544         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1545         //  r8:  T0{c0,c1}  T1{c2,c3} T2{c4,c5} T3{c6,c7}  |
1546         //  r9:  T4{c0,c1}  T5{c2,c3} T6{c4,c5} T7{c6,c7}  |
1547         // r10:  T8{c0,c1}  T9{c2,c3} T10{c4,c5} T11{c6,c7} | RC[1]
1548         //  ...
1549         // r15:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1550         //
1551         // Within a 4-lane group (e.g. T0-T3 for row 0):
1552         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1553         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1554         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0，4 次 shuffle 凑齐
1555         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}，再由 lane%4==0 128-bit store.
1556         // =====
1557 #pragma unroll
1558         for (int j = 0; j < VAL_TILE_N; ++j) {
1559             RC0[j][0] = RC[i][j][0];
1560             RC1[j][0] = RC[i][j][1];
1561             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1562             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1563             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1564             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1565             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1566             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1567         }
1568         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1569         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行)：
1570         //
1571         // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1572         // |-----|-----|-----|-----|
1573         // | 0~3   | 0       | row 0     | row 8     |
1574         // | 4~7   | 1       | row 1     | row 9     |

```

```

1575 // | 8~11 | 2 | row 2 | row 10 |
1576 // | 12~15 | 3 | row 3 | row 11 |
1577 // | 16~19 | 4 | row 4 | row 12 |
1578 // | 20~23 | 5 | row 5 | row 13 |
1579 // | 24~27 | 6 | row 6 | row 14 |
1580 // | 28~31 | 7 | row 7 | row 15 |
1581 // |-----|
1582 if (lane_id % 4 == 0) {
1583     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1584     int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1585     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1586 #pragma unroll
1587     for (int j = 0; j < VAL_TILE_N; ++j) {
1588         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1589         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1590         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1591         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1592         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1593         // 128-bit store: 一次写入 8 个 half
1594         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1595             *reinterpret_cast<float4*>(&RC0[j][0]);
1596         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1597             *reinterpret_cast<float4*>(&RC1[j][0]);
1598     }
1599 }
1600 }
1601 }
1602 }
1603
1604 // =====
1605 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1606 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1607 // =====
1608 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1609 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1610 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way)。
1611 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1612 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1613 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1614 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1615 //   rows 8~11: repeat rows 0~3 pattern
1616 //   rows 12~15: repeat rows 4~7 pattern
1617 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1618 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1619 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1620 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1621 //
1622 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1623 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1624
1625 // ---- Swizzle 辅助函数 ----
1626
1627 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1628 // i: row index; j: col index.
1629 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1630 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1631 // 用位运算展开 (kStep=8): (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1632 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1633 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1634 template <const int kColStride = 16, const int kStep = 8>
1635 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1636     // for col_stride > 16, we have to permute it using col major ZigZag order.
1637     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].

```

```

1638 static_assert(kColStride <= 16, "kColStride must <= 16");
1639 // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1640 static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1641 static_assert(kColStride % kStep == 0,
1642             "kColStride must be multiple of kStep.");
1643 if constexpr (kStep == 8) {
1644     return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1645 } else {
1646     static_assert(kStep == 4);
1647     return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1648 }
1649 }
1650
1651 // swizzle_permuted_A_j: A 矩阵专用封装 (kMmaAtomK=16, kStep=8)。
1652 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1653 // 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
1654 // 触发一次合并的memory transaction. 这里的col=16的swizzle, 相当于
1655 // TMA中的SWIZZLE_32B pattern.
1656 // -----
1657 // -col 0~16, step 8--
1658 // -----
1659 // | row 0 | (0, 8) |
1660 // | row 1 | (0, 8) |
1661 // | row 2 | (0, 8) |
1662 // | row 3 | (0, 8) |
1663 // -----
1664 // | row 4 | (8, 0) |
1665 // | row 5 | (8, 0) |
1666 // | row 6 | (8, 0) |
1667 // | row 7 | (8, 0) |
1668 // -----
1669 // | row 8 | (0, 8) |
1670 // | row 9 | (0, 8) |
1671 // | row 10 | (0, 8) |
1672 // | row 11 | (0, 8) |
1673 // -----
1674 // | row 12 | (8, 0) |
1675 // | row 13 | (8, 0) |
1676 // | row 14 | (8, 0) |
1677 // | row 15 | (8, 0) |
1678 // -----
1679 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1680 template <const int kMmaAtomK = 16>
1681 static __device__ __forceinline__ int swizzle_permuted_A_j(int i, int j) {
1682     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1683 }
1684
1685 // swizzle_permuted_B_j: B 矩阵专用封装 (与 A 相同的 pattern)。
1686 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1687 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8)。
1688 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1689 template <const int kMmaAtomK = 16>
1690 static __device__ __forceinline__ int swizzle_permuted_B_j(int i, int j) {
1691     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1692 }
1693
1694 // =====
1695 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1696 // + Register Double Buffering (VAL_TILE_K=2, BK=32 统一 tile)
1697 // =====
1698 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1699 // - VAL_TILE_K=2: 每个 BK tile 包含 2 个 MMA_K slice (BK = MMA_K * VAL_TILE_K = 32)
1700 // - BK=32 统一 tile: MMA_K=0 和 MMA_K=1 数据连续存放在同一个 BK=32 宽的 smem tile 中

```

```

1701 // - RA[2][VAL_TILE_M][4] / RB[2][VAL_TILE_N][2]: 双份寄存器乒乓切换
1702 // - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
1703 //
1704 // * smem 布局:
1705 // s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
1706 // s_b: 紧接 s_a 之后
1707 // 每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +MMA_K(=16)
1708 //
1709 // * 寄存器双缓冲时间线 (每 k 迭代内):
1710 // Step 1: G→S cp.async 预取 stage(k+K_STAGE-1) 全部 k_step (条件)
1711 // Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1712 // Step 3: for k_step=1..VAL_TILE_K-1: ldmatrix(列偏移 k_step*MMA_K)→reg[store], flip, MMA→reg[load]
1713 // Step 4: adaptive wait + __syncthreads
1714 // Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1715 //
1716 // 与 dsmem 版本的对比:
1717 // - dsmem: smem 分两个区域存放 MMA_K=0 和 MMA_K=1, smem 翻倍
1718 // - BK=32: MMA_K=0/1 连续存同一 tile 内, smem 不变, gmem 索引用 k*BK 直接定位
1719 //
1720 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1721 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1722 // Block: (256, 1, 1), 8 warps
1723 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1724           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1725           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1726           const int VAL_TILE_K = 2,
1727           const int K_STAGE = 3,
1728           const bool BLOCK_SWIZZLE = false>
1729 __global__ void __launch_bounds__(256)
1730     hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1731     // Block Swizzle: 在 grid × 维度做 swizzle, 改善 L2 cache 局部性
1732     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1733     const int by = blockIdx.y;
1734     constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1735     constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1736     constexpr int BK = MMA_K * VAL_TILE_K; // 16*2=32
1737
1738     // K_STAGE stages, each with BM×BK for A, BN×BK for B
1739     // smem: VAL_TILE_K 个 MMA_K slice 连续存放在同一个 BK=32 宽 tile 中
1740     extern __shared__ half smem[];
1741     half *s_a = smem;
1742     half *s_b = smem + K_STAGE * BM * BK;
1743     constexpr int s_a_stage_offset = BM * BK;
1744     constexpr int s_b_stage_offset = BN * BK;
1745
1746     const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1747     const int warp_id = tid / WARP_SIZE;
1748     const int lane_id = tid % WARP_SIZE;
1749     const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1750     const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1751
1752     // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1753     // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1754     int load_smem_a_m = tid / 2; // 0~127
1755     int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1756     int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1757     int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1758     int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1759     int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1760     if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1761         return;
1762
1763     // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].

```

```

1764 // CUDA中每个block能放的线程数是有上限的（warp数量有限），一般为4或者8个warps。为了能处理更大的C tile，
1765 // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
1766 // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
1767 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1768
1769 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1770 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1771
1772 // Prefetch (K_STAGE-1) stages: load all k_step slices for each early stage
1773 #pragma unroll
1774 for (int k = 0; k < (K_STAGE - 1); ++k) {
1775 #pragma unroll
1776 for (int k_step = 0; k_step < VAL_TILE_K; ++k_step) {
1777     int load_gmem_a_k = k * BK + k_step * MMA_K + load_smem_a_k;
1778     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1779     int load_gmem_b_k = k * BK + k_step * MMA_K + load_smem_b_k;
1780     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1781
1782     uint32_t load_smem_a_ptr =
1783         (smem_a_base_ptr +
1784          (k * s_a_stage_offset + load_smem_a_m * BK +
1785           k_step * MMA_K +
1786            swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1787            sizeof(half));
1788     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1789
1790     uint32_t load_smem_b_ptr =
1791         (smem_b_base_ptr +
1792          (k * s_b_stage_offset + load_smem_b_n * BK +
1793           k_step * MMA_K +
1794            swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1795            sizeof(half));
1796     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1797 }
1798 CP_ASYNC.COMMIT_GROUP();
1799 }
1800
1801 CP_ASYNC.WAIT_GROUP(K_STAGE - 2);
1802 __syncthreads();
1803
1804 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1805 uint32_t RA[2][VAL_TILE_M][4];
1806 uint32_t RB[2][VAL_TILE_N][2];
1807 int reg_st_idx = 0; // write target for ldmatrix
1808 int reg_ld_idx = 1; // read source for MMA
1809
1810 // Initial ldmatrix: load stage 0, k_step=0 (MMA_K=0 列) → reg[0]
1811 #pragma unroll
1812 for (int i = 0; i < VAL_TILE_M; ++i) {
1813     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1814     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1815     int lane_smem_a_k = (lane_id / 16) * 8;
1816     uint32_t lane_smem_a_ptr =
1817         (smem_a_base_ptr +
1818          (0 * s_a_stage_offset + lane_smem_a_m * BK +
1819           swizzle_permuted_A_j<MMA_K>(lane_smem_a_m, lane_smem_a_k)) *
1820            sizeof(half));
1821     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1822                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1823                 lane_smem_a_ptr);
1824 }
1825 #pragma unroll
1826 for (int j = 0; j < VAL_TILE_N; ++j) {

```

```

1827     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1828     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1829     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1830     uint32_t lane_smem_b_ptr =
1831         (smem_b_base_ptr +
1832          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1833           swizzle_permuted_B_j<MMA_K>(lane_smem_b_n, lane_smem_b_k)) *
1834           sizeof(half)));
1835     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1836                 lane_smem_b_ptr);
1837 }
1838
1839 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
1840 // BK = MMA_K * VAL_TILE_K = 32, 每个 k 迭代覆盖 BK=32 个 K 元素
1841 const int NUM_K_TILES = div_ceil(K, BK);
1842 #pragma unroll
1843 for (int k = 0; k < NUM_K_TILES; ++k) {
1844     int smem_sel = k % K_STAGE;
1845     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE;
1846
1847     // G→S: 条件预取 stage(k+K_STAGE-1) 全部 k_step slice
1848     if (k + K_STAGE - 1 < NUM_K_TILES) {
1849 #pragma unroll
1850         for (int k_step = 0; k_step < VAL_TILE_K; ++k_step) {
1851             int load_gmem_a_k =
1852                 (k + K_STAGE - 1) * BK + k_step * MMA_K + load_smem_a_k;
1853             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1854             int load_gmem_b_k =
1855                 (k + K_STAGE - 1) * BK + k_step * MMA_K + load_smem_b_k;
1856             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1857
1858             uint32_t load_smem_a_ptr =
1859                 (smem_a_base_ptr +
1860                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1861                   k_step * MMA_K +
1862                    swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1863                    sizeof(half)));
1864             CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1865
1866             uint32_t load_smem_b_ptr =
1867                 (smem_b_base_ptr +
1868                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1869                   k_step * MMA_K +
1870                    swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1871                    sizeof(half)));
1872             CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1873         }
1874         CP_ASYNC_COMMIT_GROUP();
1875     }
1876
1877     // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
1878     // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1879     // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1880
1881 #pragma unroll
1882     for (int k_step = 0; k_step < VAL_TILE_K; ++k_step) {
1883         reg_st_idx ^= 1; // 0 → 1; 1 → 0
1884         reg_ld_idx ^= 1; // 1 → 0; 0 → 1
1885
1886         if (k_step + 1 < VAL_TILE_K) {
1887             int smem_k_offset = (k_step + 1) * MMA_K;
1888 #pragma unroll
1889             for (int i = 0; i < VAL_TILE_M; ++i) {

```



```

1890     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1891     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1892     int lane_smem_a_k = (lane_id / 16) * 8;
1893     uint32_t lane_smem_a_ptr =
1894         (smem_a_base_ptr +
1895         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1896         smem_k_offset +
1897         swizzle_permuted_A_j<MMA_K>(lane_smem_a_m, lane_smem_a_k)) *
1898         sizeof(half));
1899     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1900         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1901         lane_smem_a_ptr);
1902 }
1903 #pragma unroll
1904 for (int j = 0; j < VAL_TILE_N; ++j) {
1905     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1906     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1907     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1908     uint32_t lane_smem_b_ptr =
1909         (smem_b_base_ptr +
1910         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1911         smem_k_offset +
1912         swizzle_permuted_B_j<MMA_K>(lane_smem_b_n, lane_smem_b_k)) *
1913         sizeof(half));
1914     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1915         lane_smem_b_ptr);
1916 }
1917 }
1918
1919 #pragma unroll
1920 for (int i = 0; i < VAL_TILE_M; ++i) {
1921 #pragma unroll
1922     for (int j = 0; j < VAL_TILE_N; ++j) {
1923         HMMA16816(RC[i][j][0], RC[i][j][1],
1924             RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
1925             RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
1926             RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
1927             RC[i][j][0], RC[i][j][1]);
1928     }
1929 }
1930 }
1931
1932 // 自适应等待: 满载期用 K_STAGE-2, 尾部排空用 0
1933 if (k + K_STAGE - 1 < NUM_K_TILES) {
1934     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1935 } else {
1936     CP_ASYNC_WAIT_GROUP(0);
1937 }
1938 __syncthreads();
1939
1940 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
1941 if (k + 1 < NUM_K_TILES) {
1942     smem_sel = (k + 1) % K_STAGE; // old smem_sel + 1
1943 #pragma unroll
1944     for (int i = 0; i < VAL_TILE_M; ++i) {
1945         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1946         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1947         int lane_smem_a_k = (lane_id / 16) * 8;
1948         uint32_t lane_smem_a_ptr =
1949             (smem_a_base_ptr +
1950             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1951             swizzle_permuted_A_j<MMA_K>(lane_smem_a_m, lane_smem_a_k)) *
1952             sizeof(half));

```

```

1953     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1954                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1955                 lane_smem_a_ptr);
1956 }
1957 #pragma unroll
1958 for (int j = 0; j < VAL_TILE_N; ++j) {
1959     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1960     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1961     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1962     uint32_t lane_smem_b_ptr =
1963         (smem_b_base_ptr +
1964          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1965           swizzle_permuted_B_j < MMA_K > (lane_smem_b_n, lane_smem_b_k)) *
1966           sizeof(half));
1967     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1968                 lane_smem_b_ptr);
1969 }
1970 }
1971 }
1972
1973 // Epilogue: 复用 RA 寄存器做 warp shuffle → 128-bit collective store
1974 for (int i = 0; i < VAL_TILE_M; ++i) {
1975 #pragma unroll
1976     for (int j = 0; j < VAL_TILE_N; ++j) {
1977         RA[0][j][0] = RC[i][j][0];
1978         RA[1][j][0] = RC[i][j][1];
1979         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1980         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1981         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1982         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1983         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1984         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1985     }
1986     if (lane_id % 4 == 0) {
1987         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1988         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1989 #pragma unroll
1990         for (int j = 0; j < VAL_TILE_N; ++j) {
1991             int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1992             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1993             int store_gmem_c_addr_0 =
1994                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1995             int store_gmem_c_addr_1 =
1996                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1997             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1998                 *reinterpret_cast<float4*>(&RA[0][j][0]);
1999             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2000                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2001         }
2002     }
2003 }
2004 }
2005
2006 // =====
2007 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2008 // =====
2009 // 面试要点 (CuTe vs 手写 MMA PTX) :
2010 //   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2011 //     Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2012 //   - 与手写 MMA PTX (Phase 7b) 的对比:
2013 //     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2014 //     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2015 //       cute::copy / cute::gemm 自动展开为高效指令序列

```

```

2016 // - 核心抽象 ("CuTe 五要素") :
2017 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2018 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2019 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2020 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2021 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2022 //
2023 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2024 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2025 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2026 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2027 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2028 //
2029 // 调参口诀 (面试速记) :
2030 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2031 //   Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2032 //   重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2033 //     1. 连续的 2^M 个元素组成二维空间中的一个基本元素;
2034 //     2. 连续的 2^S 个基本元素组成一行;
2035 //     3. 二维空间包含 2^B 行;
2036 //     4. 对二维坐标做列置换: icol' = irow ^ icol;
2037 //     5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2038 //   因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2039 //   CuTe 的位级实现等价于:
2040 //     offset' = offset ^ ((offset & YYY) >> S),
2041 //     YYY = ((1 << B) - 1) << (M + S)。
2042 //   对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2043 //   二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2044 //   一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2045 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2046 //   EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2047 //   与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
2048 //
2049 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2050 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2051 //   - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2052 //   - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2053 //
2054 // ★ 本实现的 Tile 配置:
2055 //   - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2056 //   - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2057 //   - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2058 //     BN=256 不是由 "8x2x16" 直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2059 //   - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2060 //     的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2061 //   - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
2062 //   - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2063 //
2064 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2065 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2066 // =====
2067 //
2068 #if defined(NOTES_V2_ENABLE_CUTE)
2069 //
2070 #include <cute/tensor.hpp>
2071 //
2072 // =====
2073 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2074 // =====
2075 // TN 布局: C[MxN] = A[MxK] × B^T[NxK]
2076 //   - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [KxN])
2077 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2078 //

```

```

2079 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链):
2080 // 1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
2081 // 2. 主循环 over K tiles:
2082 // a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2083 // b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2084 // c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2085 // d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2086 // 3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store)。
2087 //
2088 // 面试对比 — 手写 MMA vs CuTe 代码量:
2089 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2090 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2091 template <typename T, int BM, int BN, int BK, int kStage, typename TiledMMA,
2092          typename G2SCopyA, typename G2SCopyB, typename SmemLayoutA,
2093          typename SmemLayoutB, typename SmemLayoutC, typename S2RCopyAtomA,
2094          typename S2RCopyAtomB, typename R2SCopyAtomC, typename S2GCopyAtomC,
2095          typename S2GCopyC, const bool BlockSwizzle = false>
2096 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2097          int n, int k) {
2098     using namespace cute;
2099
2100     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2101     extern __shared__ T shm_data[];
2102     T *Ashm = shm_data;
2103     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2104
2105     int idx = threadIdx.x;
2106     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2107     int ix = ((int)BlockSwizzle) * blockDim.z * gridDim.x + blockIdx.x;
2108     int iy = blockIdx.y; // M 方向 block 索引
2109     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2110     // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2111     if (iy * BM >= m || ix * BN >= n) return;
2112
2113     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2114     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2115     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2116                             make_stride(k, Int<1>{}));
2117     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2118                             make_stride(k, Int<1>{}));
2119     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2120                             make_stride(n, Int<1>{}));
2121
2122     // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2123     // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2124     // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2125     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2126     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2127     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2128
2129     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2130     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2131     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2132
2133     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2134     TiledMMA tiled_mma;
2135     auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2136     auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2137     auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2138     auto tCrD = thr_mma.partition_fragment_C(gD);           // (MMA, MMA_M, MMA_N)
2139     cclear(tCrD); // 累加器清零
2140
2141     // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。

```

```

2142 G2SCopyA g2s_tiled_copy_a;
2143 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2144 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2145 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2146
2147 G2SCopyB g2s_tiled_copy_b;
2148 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2149 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2150 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2151
2152 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2153 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2154 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2155 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2156 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2157 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2158
2159 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2160 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2161 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2162 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2163
2164 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2165 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2166 #pragma unroll
2167 for (int istage = 0; istage < kStage - 1; ++istage) {
2168     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2169               tAsA_copy(_, _, _, istage));
2170     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2171               tBsB_copy(_, _, _, istage));
2172     cp_async_fence();
2173     ++itile_to_read;
2174     ++ismem_write;
2175 }
2176 cp_async_wait<kStage - 2>();
2177 __syncthreads();
2178
2179 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2180 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2181 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2182 int num_k_tiles = k / BK;
2183 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2184 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2185 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2186
2187 #pragma unroll 1
2188 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2189     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2190     //
2191     // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2192     // tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2193     // - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2194     //   所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2195     // - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2196     //   同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2197     //   在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2198     //   无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2199     //   for (i=0; i<VAL_TILE_M; ++i)
2200     //       for (j=0; j<VAL_TILE_N; ++j)
2201     //           HMMMA16816(RC[i][j][0], RC[i][j][1], ...);
2202     //
2203     // CUTLASS 推导链 (mma_atom.hpp) :
2204     // make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMNK

```

```

2205 // → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2206 // → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2207 // 做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2208 // K 被分为 BK/kMmaPK 个 perm-K slice
2209 // 随后按 AtomShape_MNK<2> (MMA_K_atom) 做 zipped_divide
2210 // 每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2211 // → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2212 // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2213 int num_k_steps = size<2>(tCrA);
2214
2215 #pragma unroll
2216 for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2217     int k_step_next = (k_step + 1) % num_k_steps;
2218
2219     if (k_step == num_k_steps - 1) {
2220         // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2221         cp_async_wait<kStage - 2>();
2222         __syncthreads();
2223         ismem_read = (ismem_read + 1) % kStage;
2224     }
2225
2226     // S→R: 加载下一组 A/B fragment 到寄存器。
2227     //
2228     // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2229     // cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2230     // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**：
2231     // 当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2232     // 让 S→R 延迟与 MMA 计算重叠。
2233     // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2234     cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2235               tCrA_view(_, _, k_step_next));
2236     cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2237               tCrB_view(_, _, k_step_next));
2238
2239     if (k_step == 0) {
2240         // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2241         if (itile_to_read < num_k_tiles) {
2242             cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2243                       tAsA_copy(_, _, _, ismem_write));
2244             cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2245                       tBsB_copy(_, _, _, ismem_write));
2246             ++itile_to_read;
2247             ismem_write = (ismem_write + 1) % kStage;
2248         }
2249         cp_async_fence();
2250     }
2251
2252     // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2253     // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2254     cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2255 }
2256 }
2257
2258 // Epilogue: D 寄存器 → shared memory → global memory。
2259 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2260 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2261 // sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2262 // 只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2263 // 解释这同一段地址。当前 wrapper 的固定配置下:
2264 // 一个 A stage: 128 × 32 = 4096 个 half;
2265 // C scratchpad: 32 × 32 × 4 = 4096 个 half。
2266 // 两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2267 // 是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把

```



```

2268 // C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2269 // 检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2270 //
2271 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2272 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2273 // 用它写 C, S2G 也用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2274 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2275
2276 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2277 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2278 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2279 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2280 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2281 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2282 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2283 //
2284 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2285 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2286 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2287 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2288 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2289 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2290 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2291 // 角色, 而不是另一次 memory transfer。
2292 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2293 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2294 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2295 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2296 // MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2297 // → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2298 // → make_tiled_copy_C 的 tiler = (32, 32)
2299 // → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2300 // → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2301 // → 结果 shape 的 M/N remainder = _1, _1
2302 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2303 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2304 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2305 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2306
2307 // S2G TiledCopy: shared memory → global memory (128-bit store)
2308 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2309 S2GCopyC s2g_tiled_copy_c;
2310 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2311 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2312 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2313 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2314 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2315 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2316 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2317 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2318
2319 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2320 // group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2321 // → Tensor(CPY, (CPY_M, CPY_N))
2322 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。
2323 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系, 只是现在可以用一个坐标
2324 // i+j 访问这个组合 mode; 因此这里的 group_modes 更准确地说“合并 mode”,
2325 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2326 // 从 type/shape 结构看, 结果确实是 Tensor(CPY, (CPY_M, CPY_N)); 但 grouped
2327 // mode 的 cardinality 是两个子 mode 的乘积, 因此 size<1>(…) 可作为一个标量
2328 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2329 // 的线性 logical coordinate, 不是在 C++ 中对二维 tuple 做加法, 更不是 raw
2330 // pointer offset; 最终物理地址仍由各自 Tensor 的 grouped layout 计算。

```



```

2331 //
2332 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2333 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment, 原始形状为 (CPY, CPY_M, CPY_N);
2334 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置, 形状与源 Tensor 对齐;
2335 //   - 两者使用相同的 [1,3] 合并后, 源/目的 Tensor 仍保持相同的坐标空间,
2336 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2337 //
2338 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode, 因为它们还需要显式保留
2339 // shared-memory C scratchpad 的 pipeline 维度; step=size<3>(tCsC_r2s) 表示
2340 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2341 // 按 step 前进, 内层 j 选择当前 pipeline slot: 寄存器先写入 sC(_,0,j),
2342 // 再从同一个 slot 写回 global memory。
2343 // 当前 kSmemLayoutCBatch=4, 因此 step=4。代码未对 i+j 做尾部 predication,
2344 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除; 不应将
2345 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2346 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2347 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2348
2349 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2350 // 双层循环的语义 (注意 step 与 CPY_N 无关):
2351 //   group_modes<1,3> 之后, tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2352 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N, 即该线程需处理的 C fragment 总数。
2353 //   外循环以 step=4 为步长, 内循环 j=0..3 每次处理 1 个 fragment, 将其写入
2354 //   第 j 号 C scratchpad pipe slot, 再读出写回 global。
2355 //
2356 //   这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2357 //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2358 //   通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2359 //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。
2360 //   唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2361 //
2362 //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2363 //   各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2364 //   写入和读取命中同一段 shared memory。
2365 #pragma unroll
2366 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2367     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2368 #pragma unroll
2369     for (int j = 0; j < step; ++j) {
2370         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2371         // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2372         // 成输出元素类型 T 的临时 payload。
2373         //
2374         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2375         //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2376         // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2377         // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2378         //
2379         // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2380         // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2381         // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2382         // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2383         // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2384         // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2385         // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2386         // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2387         //
2388         // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2389         // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2390         // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由 R2R copy 承担, 而是
2391         // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2392         // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2393         // 保持同源实现的通用 staging 写法。

```

```

2394 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2395 cute::copy(tCrC_r2sx(_, i + j), t);
2396 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2397 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2398 //
2399 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2400 //   r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>,T>
2401 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2402 //   → src(t) 与 dst(tCsC_r2s(_,0,0,j)) 都是 rank-1 → 直接 copy_atom.call
2403 //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2404 //   走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2405 //   → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2406 //
2407 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2408 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2409 //   - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2410 //   → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2411 //   - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2412 //   → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2413 //   计算了当前线程每个 val 在 smem 中的物理 offset
2414 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2415 // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2416 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2417 }
2418 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2419 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2420 __syncthreads();
2421
2422 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2423 #pragma unroll
2424 for (int j = 0; j < step; ++j) {
2425   // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2426   // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2427   // (如 st.global.v4.f32 或 st.global.b128)。
2428   //
2429   // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2430   //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>,T>
2431   //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2432   //   → src=tCsC_s2g(_,0,0,j) 与 dst=tCgC_s2gx(_,i+j) 都是 rank-1
2433   //   → 直接 copy_atom.call(src, dst)
2434   //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2435   //   → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2436   //   → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2437   //
2438   // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2439   //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2440   //   用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2441   //   在 sC (shared memory) 中的物理 offset。
2442   //   S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2443   //   = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2444   //   因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2445   //   M/N remainder。
2446   //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2447   //   → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2448   //   因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2449   //   repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2450   // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2451   // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2452   cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2453 }
2454 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2455 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2456 __syncthreads();

```

```

2457 }
2458
2459 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2460 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2461 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2462 // CuTe 版没有把这些 lane 映射写在 kernel 源码中: retile_S 表达 accumulator
2463 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2464 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2465 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2466 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2467 }
2468
2469 // =====
2470 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2471 // =====
2472 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2473 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2474 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2475 //
2476 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2477 //
2478 // 面试常问: 「CuTe 为什么比手写 PTX 简洁? 」
2479 // 答: 手写需要:
2480 //   1) 手动计算每线程的 smem 地址偏移
2481 //   2) 手动计算 ldmatrix 的 lane 映射
2482 //   3) 手动插入 swizzle XOR 到每个地址计算点
2483 //   4) 手动做 epilogue 的 warp shuffle 编排
2484 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2485 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2486 // =====
2487 template <typename T, const int Stages = 2>
2488 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2489     using namespace cute;
2490
2491     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2492     //
2493     // kernel 用法对照:
2494     //   BM=128:   local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2495     //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2496     //   BN=256:   local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2497     //             local_tile(D, ...) → gD 的列方向
2498     //   BK=32:    num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2499     //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2500     //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2501     auto BM = Int<128>{};
2502     auto BN = Int<256>{};
2503     auto BK = Int<32>{};
2504     auto KStage = Int<Stages>{};
2505     auto kSmemLayoutCBatch = Int<4>{};
2506
2507     // — SmemLayoutA / SmemLayoutB —
2508     // kernel 用法:
2509     // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2510     // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2511     // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2512     //
2513     // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2514     //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2515     //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2516     //   - B=3: 共有 2^3=8 行。
2517     // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2518     // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2519     // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。

```

```

2520 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2521 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2522 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2523 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2524 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2525 // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2526 using SmemLayoutAtom = decltype(composition(
2527     Swizzle<3, 3, 3>{},
2528     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2529         make_stride(Int<BK>{}, Int<1>{}))));
2530 using SmemLayoutA = decltype(tile_to_shape(
2531     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2532 using SmemLayoutB = decltype(tile_to_shape(
2533     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2534
2535 // — MMA (TiledMMA) —
2536 // kernel 用法:
2537 // TiledMMA tiled_mma;
2538 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2539 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2540 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2541 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2542 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2543 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2544 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2545 //
2546 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2547 // → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2548 // → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2549 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2550 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2551 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2552 using mma_traits = MMA_Traits<mma_op>;
2553 using mma_atom = MMA_Atom<mma_traits>;
2554 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2555
2556 static constexpr int kMmaEURepeatM = 2;
2557 static constexpr int kMmaEURepeatN = 2;
2558 static constexpr int kMmaEURepeatK = 1;
2559 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2560 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2561 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2562 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2563
2564 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2565     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2566 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2567 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2568
2569 // — G2SCopyA / G2SCopyB —
2570 // kernel 用法:
2571 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2572 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,istage), tAsA_copy(_,_,istage));
2573 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,istage), tBsB_copy(_,_,istage));
2574 // G→S: 128-bit cp.async. ThrLayout{32,4} × ValLayout{1,8}:
2575 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2576 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2577 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2578 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2579 using G2SCopyA = decltype(make_tiled_copy(
2580     g2s_copy_atom{},
2581     make_layout(make_shape(Int<32>{}, Int<4>{}),
2582         make_stride(Int<4>{}, Int<1>{}))),

```

```

2583     make_layout(make_shape(Int<1>{}, Int<8>{})));
2584 using G2SCopyB = G2SCopyA;
2585
2586 // — S2RCopyAtomA / S2RCopyAtomB —
2587 // kernel 用法:
2588 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2589 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2590 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2591 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2592 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2593 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout。
2594 using s2r_copy_op = SM75_U32x4_LDSM_N;
2595 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2596 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2597 using S2RCopyAtomA = s2r_copy_atom;
2598 using S2RCopyAtomB = s2r_copy_atom;
2599
2600 // — SmemLayoutC —
2601 // kernel 用法:
2602 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2603 // 复用当前 A stage 的空间作为 C scratchpad
2604 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2605 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2606 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2607 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2608 // 的 kStage K-tile pipeline。
2609 using SmemLayoutAtomC = decltype(composition(
2610     Swizzle<3, 3, 3>{},
2611     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2612         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2613 using SmemLayoutC = decltype(tile_to_shape(
2614     SmemLayoutAtomC{},
2615     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2616
2617 // — R2SCopyAtomC —
2618 // kernel 用法:
2619 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2620 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2621 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2622 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2623 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2624 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2625
2626 // — S2GCopyC —
2627 // kernel 用法:
2628 // S2GCopyC s2g_tiled_copy_c;
2629 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2630 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared-global wide store。
2631 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2632 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2633 using S2GCopyAtomC = Copy_Atom<UniversalCopy<uint128_t>, T>;
2634 using S2GCopyC = decltype(make_tiled_copy(
2635     S2GCopyAtomC{},
2636     make_layout(make_shape(Int<32>{}, Int<4>{}),
2637         make_stride(Int<4>{}, Int<1>{})),
2638     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2639
2640 // — Grid/Block 配置 —
2641 // kernel 用法:
2642 // int idx = threadIdx.x; // 0..127
2643 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2644 // int iy = blockIdx.y; // M 方向 CTA 坐标
2645 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护

```



```

2646 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2647 // M % BM == 0, N % BN == 0, K % BK == 0。
2648 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2649 constexpr bool kBlockSwizzle = false;
2650 constexpr int kSwizzleStride = 2048;
2651 int BX = (N + BN - 1) / BN;
2652 int BY = (M + BM - 1) / BM;
2653 int BZ = kBlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2654 BX = kBlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2655 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2656 dim3 grid(BX, BY, BZ);
2657
2658 // — Dynamic Shared Memory 大小 —
2659 // kernel 用法:
2660 // extern __shared__ T shm_data[];
2661 // T *Ashm = shm_data;
2662 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2663 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2664 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2665 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2666 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2667 // 能放进一个 A stage (128*32 = 4096)。
2668 static constexpr int shm_size_AB =
2669     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2670 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2671 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2672     "C shared memory must fit within one A pipe");
2673 static constexpr int kShmSize =
2674     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2675
2676 cudaFuncSetAttribute(
2677     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2678         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2679         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2680         S2GCopyAtomC, S2GCopyC, kBlockSwizzle>,
2681     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2682
2683 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2684     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2685     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2686     kBlockSwizzle>
2687     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2688 }
2689
2690 #endif /* NOTES_V2_ENABLE_CUTE */
2691
2692 // =====
2693 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2694 // =====
2695 // 面试要点 (WGMMMA vs MMA 对比):
2696 // - MMA: warp 级 (32 threads), 同步执行
2697 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2698 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4x16=64倍)
2699 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2700 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2701 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2702
2703 // ---- WGMMMA 辅助函数 ----
2704 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7):
2705 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2706 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2707 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2708 //   N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),

```

```

2709 // 都是 "wait until only N or fewer groups are pending"。
2710 #define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2711 #define WGMMA_COMMIT_GROUP() \
2712     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2713 #define WGMMA_WAIT_GROUP(n) \
2714     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2715
2716 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
2717 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2718 //   encoded = (x & 0x3FFFF) >> 4
2719 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2720 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2721 // 可省略以扩大可表示范围。
2722 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2723 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2724
2725 // make_smem_desc: 构造 WGMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2726 // 硬件 WGMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2727 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2728 //
2729 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2730 // -----
2731 // | 位域           | 范围       | 大小 | 含义
2732 // |-----|-----|-----|-----
2733 // | start_address   | [0, 14)    | 14   | smem 基址编码
2734 // | (unused)        | [14, 16)   | 2    | -
2735 // | leading_byte_offset | [16, 30)   | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2736 // | (unused)        | [30, 32)   | 2    | -
2737 // | stride_byte_offset | [32, 46)   | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2738 // | (unused)        | [46, 49)   | 3    | -
2739 // | base_offset     | [49, 52)   | 3    | swizzle pattern 偏移
2740 // | (unused)        | [52, 62)   | 10   | -
2741 // | layout_type     | [62, 64)   | 2    | swizzle 模式
2742 // -----
2743 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2744 //
2745 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2746 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2747 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2748 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2749 // - 这是 stride_byte_offset=1024 的来源
2750 //
2751 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1),
2752 //   此处填 16 仅为占位, 编码后为 1。
2753 //
2754 // - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。
2755 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2756 //
2757 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2758 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2759 __device__ inline uint64_t make_smem_desc(half *ptr) {
2760     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2761     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2762     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2763
2764     // 从全零开始: 所有位域初始化为 0。
2765     uint64_t desc = 0x0000000000000000;
2766
2767     // — bits [0, 14): start_address —
2768     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit),
2769     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2770     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2771     desc |= SMEM_DESC_ENCODE(addr);

```



```

2772
2773 // — bits [16, 30): leading_byte_offset —
2774 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2775 // << 16 将编码值放到 bits [16, 30)。
2776 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1),
2777 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2778 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2779 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2780 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2781 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2782
2783 // — bits [32, 46): stride_byte_offset —
2784 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2785 // << 32 将编码值放到 bits [32, 46)。
2786 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2)。
2787 // 1024 的来源 (K-Major + 128B swizzle + half):
2788 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2789 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2790 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组)。
2791 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2792
2793 // — bits [62, 64): layout_type (swizzle mode) —
2794 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2795 // 1 = 128B swizzle mode。
2796 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2797 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2798 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2799 desc |= 1llu << 62;
2800
2801 return desc;
2802 }
2803
2804 // ---- WGMMA PTX 指令宏 ----
2805 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2806 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
2807 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2808 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
2809 // 每线程 64 half = 32 uint32。
2810 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2811 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2812 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2813 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2814 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2815 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2816                                     TransB) \
2817 { \
2818     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2819     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2820     asm volatile( \
2821         "{\n" \
2822         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2823         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2824         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2825         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2826         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2827         "%32, " \
2828         "%33, " \
2829         "%34, %35, %36, %37, %38;\n" \
2830         "}\n" \
2831         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2832         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2833         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2834         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \

```

```

2835         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2836         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2837         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2838         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2839     : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaledD)), \
2840     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2841     "n"(int32_t(TransB))); \
2842 }
2843
2844 // ---- TMA Shared Memory Layout ----
2845 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2846 //
2847 // 每个 stage 包含两块 smem:
2848 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2849 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2850 //
2851 // ★ 关键区别 — WGMMMA 的 B 物理存储 vs 逻辑视图:
2852 //
2853 //   上层数据: 源矩阵 BT [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2854 //   物理存储: TMA 将 BT 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2855 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2856 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2857 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2858 //   逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2859 //               通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2860 //               实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2861 //               swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2862 //
2863 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2864 //       128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2865 //       stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2866 //                               = 8 rows × (BK=64) halves × 2 bytes = 1024。
2867 //       这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2868 //
2869 //   与 MMA(TN) 的对比:
2870 //       MMA (TN): smem 存 BT [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2871 //       WGMMMA:   smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2872 //       简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2873 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2874     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2875     // B tile 笔记:
2876     //   - 物理存储: TMA 从 BT[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2877     //   - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2878     //       以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2879     //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
2880     //       但写 BN×BK 更能体现物理存储形态
2881     alignas(128) half B[BN * BK * QSIZE];
2882 };
2883
2884 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2885 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
2886 //
2887 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2888 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2889 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2890 //
2891 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2892 //       将 A/B tile 从 HBM 搬到 shared memory。
2893 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2894 //
2895 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2896 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2897 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖

```

```

2898 //
2899 // 本节重点理解：
2900 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令，一次提交
2901 // 即可搬运整个 2D tile，无需所有线程参与。
2902 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2903 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时，
2904 // Producer 可以搬运 stage (qidx+1)，隐藏 HBM→SMEM 延迟。
2905 //
2906 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比)：
2907 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16，是 MMA 的 64 倍)
2908 //   K Tile:           BK=64，每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
2909 //   M Tile:           BM=128，每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
2910 //   N Tile:           BN=128，单个 WGMMMA_N=128 即可覆盖，无需在 N 方向分块
2911 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
2912 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
2913 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
2914 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
2915 //
2916 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步，确保 HBM→SMEM
2917 // 的延迟被计算完全掩盖。
2918 //
2919 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2920 // - grid.z = S 个 swizzle 分区，将连续 block 打散到不同 N 区域改善 L2 命中
2921 // Block: (256, 1, 1), 2 warpgroups
2922 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2923 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
2924           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
2925           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
2926           const bool BLOCK_SWIZZLE = false>
2927 __global__ void __launch_bounds__(NUM_THREADS)
2928   hgemm_wgmma_stages_tn(
2929     int M, int N, int K, half *C,
2930     const CUtensorMap *__restrict__ tensorMapA,
2931     const CUtensorMap *__restrict__ tensorMapB) {
2932   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建；
2933   // 对 row-major [M, K] 矩阵，TMA shape 参数写的是 (K, M) 而不是 (M, K)，
2934   // 也就是TMA descriptor中把连续的维度写在最内层，非连续的维度写在最外层。
2935   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体，
2936   // 不展开宿主侧 create_tensor_map 细节。
2937
2938   // Block Swizzle: 在 grid x 维度做 swizzle，改善 L2 cache 局部性
2939   // bx = blockIdx.z * gridDim.x + blockIdx.x，将相邻 block 打散到不同 N 区域
2940   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
2941   const int by = blockIdx.y;
2942   // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2943   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer，剩余都是 consumer
2944   constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
2945   // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2946   // 当只有一个 consumer 时，它负责全部 BM=128 行
2947   constexpr int B_WG_M = BM / num_consumers; // 128
2948
2949   // 边界检查: 确保当前 block 不超出 M/N 范围
2950   if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2951     return;
2952
2953   // ---- Shared Memory 分配 ----
2954   // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2955   // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2956   extern __shared__ __align__(128) uint8_t smem_AB[];
2957   WgmmaSMem<BM, BN, BK, K_STAGE> &s =
2958     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_AB);
2959   half *s_a = s.A;
2960   half *s_b = s.B;

```

```

2961 // ---- cuda::barrier 初始化 ----
2962 //
2963 // ★ Barrier 机制速查 (arrive / wait 核心语义) :
2964 //
2965 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
2966 //
2967 //   init(bar, N): 设置 arrive_count = N。
2968 //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2969 //
2970 //   arrive(): 线程声明"我已到达此 barrier 点"。
2971 //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
2972 //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
2973 //
2974 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
2975 //       - 即: 等待 phase 翻转。
2976 //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
2977 //                           (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
2978 //
2979 //
2980 // 典型用法: b.wait(b.arrive())
2981 //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
2982 //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
2983 //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
2984 //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
2985 //
2986 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129) :
2987 //
2988 //   Producer (1 thread)           Consumer (128 threads)
2989 //   -----
2990 //   C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
2991 //   ┌ for each k_tile: ─┐         ┌ for each k_tile: ─┐
2992 //   │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q]) │
2993 //   │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪   │
2994 //   │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算   │
2995 //   │   ↓ 异步拷贝             │ │ C3: wait WGMMMA 完成 │
2996 //   │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q]) │
2997 //   │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完 │
2998 //   └──────────────────┘         └──────────────────┘
2999 //
3000 // 关键不变式 (invariant) :
3001 //   - Producer 不会覆盖 Consumer 正在读的 stage
3002 //   - Consumer 不会读 Producer 还没写完的 stage
3003 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
3004 //
3005 // 每个 stage 有两个 barrier:
3006 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3007 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3008 //
3009 //   arrive_count = 128 (consumer) + 1 (producer) = 129:
3010 //   每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3011 #pragma nv_diag_suppress static_var_with_dynamic_init
3012 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
3013 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
3014 #pragma nv_diag_default static_var_with_dynamic_init
3015
3016 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3017 const int num_blocks_k = K / BK;
3018 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
3019 const int tid = threadIdx.x % 128;    // 0~127 within warpgroup
3020
3021 // 初始化 barriers (仅 thread 0 执行)
3022 if (threadIdx.x == 0) {
3023     for (int i = 0; i < K_STAGE; ++i) {

```

```

3024 // arrive_count = 129: 128 consumer + 1 producer
3025 // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内) ,
3026 // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3027 // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3028 init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
3029 init(&empty[i], num_consumers * 128 + 1); // same
3030 }
3031 // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3032 // 对 async proxy (TMA/WGMMMA) 可见。
3033 cuda::device::experimental::fence_proxy_async_shared_cta();
3034 }
3035 __syncthreads();
3036
3037 // =====
3038 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3039 // =====
3040 //
3041 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
3042 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
3043 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
3044 //
3045 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
3046 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
3047 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
3048 //
3049 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
3050 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
3051 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
3052 //
3053 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
3054 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead) 。
3055 // =====
3056 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3057 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
3058 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
3059 // =====
3060 if (wg_idx == 0) {
3061     if (tid == 0) {
3062         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3063         int qidx = 0;
3064         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3065             if (qidx == K_STAGE)
3066                 qidx = 0;
3067
3068             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
3069             //
3070             // 模式 empty[qidx].wait(empty[qidx].arrive()):
3071             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
3072             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3073             //     128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
3074             //     → phase 立即翻转。
3075             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3076             //     由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
3077             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3078             //
3079             // 语义: Producer 说"我准备好写 stage qidx 了, Consumer 用完没有? "
3080             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3081             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3082             empty[qidx].wait(empty[qidx].arrive());
3083
3084             // Step P2: 提交 TMA 2D 拷贝指令
3085             // cp_async_bulk_tensor_2d_global_to_shared:
3086             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)

```



```

3087 // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3088 // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3089 // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3090 //
3091 // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
3092 // 无需线程逐元素搬运, 零寄存器开销。
3093 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3094 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3095     &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
3096     full[qidx]);
3097
3098 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3099 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3100     &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
3101     full[qidx]);
3102
3103 // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
3104 //
3105 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3106 //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3107 //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3108 //   - Phase 翻转条件:
3109 //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3110 //       (b) 所有声明的 async 字节已写入 smem
3111 //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3112 //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3113 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
3114     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
3115 }
3116 }
3117 }
3118 // =====
3119 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3120 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3121 // 所有 128 个线程 (4 warps) 全部参与。
3122 // =====
3123 else {
3124     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3125     //
3126     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3127     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
3128     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3129     //
3130     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3131     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
3132     for (int i = 0; i < K_STAGE; ++i) {
3133         [[maybe_unused]] auto token = empty[i].arrive();
3134     }
3135
3136     // 累加器寄存器声明
3137     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
3138     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3139     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3140     //   - d[*][g][*]: N 方向第 g 组 16 列
3141     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3142     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3143     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3144     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
3145
3146     int qidx = 0;
3147     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3148     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3149         if (qidx == K_STAGE)

```



```

3150     qidx = 0;
3151
3152     // Step C1: 等待 TMA 数据就绪 (full 信号)
3153     //
3154     // 模式 full[qidx].wait(full[qidx].arrive()):
3155     //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3156     //   - 当前 phase 累积: 128/129, phase 尚未翻转。
3157     //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3158     //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3159     //
3160     // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有? "
3161     full[qidx].wait(full[qidx].arrive());
3162
3163     // Step C2: 发射 WGMMMA 指令序列
3164     //
3165     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3166     // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3167     //
3168     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3169     //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3170     //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3171     //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3172     //
3173     // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3174     //   D[64×128] += A[64×16] × B[16×128]
3175     //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3176     //   ScaleD=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都使用 ScaleD=1。
3177     WGMMMA_FENCE();
3178
3179     // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
3180     // 每个 atom 处理 64 行 M 方向数据。
3181     #pragma unroll
3182     for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
3183         // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
3184         // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
3185         half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
3186
3187         // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
3188         // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3189         #pragma unroll
3190         for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
3191             // 第 k_it 次 WGMMMA:
3192             //   A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
3193             //   B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
3194             //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3195             //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3196             //   ScaleD=1: 累加 (K 维迭代需要累积结果)
3197             WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
3198                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
3199                                     1, // ScaleD=1: accumulate (不清零)
3200                                     1, 1, 0, 0);
3201         }
3202     }
3203
3204     // Step C3: 提交并等待 WGMMMA 完成
3205     //
3206     // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3207     //   将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3208     //   类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3209     //
3210     // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3211     //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3212     //   ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :

```

```

3213 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
3214 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3215 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3216 WGMMMA_COMMIT_GROUP();
3217 WGMMMA_WAIT_GROUP(0);
3218
3219 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
3220 //
3221 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
3222 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3223 //
3224 // 语义: Consumer 说"stage qidx 我已经读完了, 你可以放心覆盖了。"
3225 [[maybe_unused]] auto token = empty[qidx].arrive();
3226 }
3227
3228 // =====
3229 // Epilogue: 将寄存器中的累加结果写回 global memory C
3230 // =====
3231 //
3232 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3233 //
3234 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3235 // 这些 half 分布在 d[2][8][4] 数组中:
3236 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3237 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3238 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3239 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3240 //
3241 // 其中:
3242 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3243 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3244 //
3245 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3246 // +-----+-----+
3247 // | reg[0] | reg[2] | <- rows [row, row+8)
3248 // |(col,+2)|(col+8)|
3249 // +-----+-----+
3250 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3251 // |(col,+2)|(col+8)|
3252 // +-----+-----+
3253 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3254 //
3255 // 三维遍历:
3256 // m_it=0: rows 0~63, m_it=1: rows 64~127
3257 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3258 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3259 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3260
3261 const int lane = tid % 32;
3262 const int warp = tid / 32;
3263 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3264 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3265 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3266 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3267 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3268 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3269 const int row = warp * 16 + lane / 4;
3270 // block_C: 当前 C tile 在 global memory 中的起始地址
3271 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3272 half *block_C = C + by * BM * N + bx * BN;
3273
3274 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3275 #pragma unroll

```

```

3276     for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
3277         int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
3278     #pragma unroll
3279         for (int g = 0; g < WGMMMA_N / 16; ++g) {
3280             int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3281             // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3282             // 左上象限: (row, col)
3283             *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
3284                 d[m_it][g][0];
3285             // 左下象限: (row+8, col)
3286             *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
3287                 d[m_it][g][1];
3288             // 右上象限: (row, col+8)
3289             *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
3290                 d[m_it][g][2];
3291             // 右下象限: (row+8, col+8)
3292             *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
3293                 d[m_it][g][3];
3294         }
3295     }
3296 }
3297 }
3298
3299 #if (defined(NOTES_V2_ENABLE_WGMMMA))
3300 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3301 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3302 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3303 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3304 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3305 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3306 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3307 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3308 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3309 //
3310 // ★ gmem_prob_stride + 1 的含义:
3311 //   语法层面 — 数组名退化 + 指针算术:
3312 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3313 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3314 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3315 //   语义层面 — 跳过隐式的最内维 stride:
3316 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3317 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3318 //     固定等于 sizeof(element_type), 不需要显式传入。
3319 //     对于 tensorRank=2 的 FP16 矩阵:
3320 //       dim 0 (内维, K): stride = sizeof(half) ← 隐式, API 不需要
3321 //       dim 1 (外维, M): stride = sizeof(half)*K ← 需要传给 API
3322 //     gmem_prob_stride 数组的布局是内维在前:
3323 //       [0] = sizeof(half) ← 内维, 不传
3324 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3325 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3326 //
3327 // ★ smem_box_stride 为什么不需要 +1?
3328 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3329 //   最内维 stride 也必须显式传入 (不隐式)。
3330 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3331 //   都是连续存放的, 维度间没有 padding/stride。
3332 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3333 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3334 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3335 //   两个 stride 值, 不需要跳过任何一个。
3336 //
3337 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3338

```

```

3339 template <int BlockMajorSize, int BlockMinorSize>
3340 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3341                                               half *gmem_ptr,
3342                                               int blocks_height,
3343                                               int blocks_width) {
3344     void *gmem_address = (void *)gmem_ptr;
3345     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3346                                     (uint64_t)BlockMajorSize * blocks_height,
3347                                     1, 1, 1};
3348     uint64_t gmem_prob_stride[5] = {
3349         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3350     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3351                                    uint32_t(BlockMajorSize), 1, 1, 1};
3352     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3353     CUresult result = cuTensorMapEncodeTiled(
3354         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3355         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3356         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3357         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3358     if (result != CUDA_SUCCESS)
3359         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3360 }
3361
3362 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
3363     half *src, int blocks_height, int blocks_width) {
3364     CUtensorMap *tma_map_d;
3365     cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
3366     CUtensorMap tma_map_host;
3367     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3368     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
3369               cudaMemcpyHostToDevice);
3370     return tma_map_d;
3371 }
3372 #endif /* NOTES_V2_ENABLE_WGMMA */
3373
3374 // =====
3375 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
3376 // =====
3377 // 面试题点 (FlashAttention 算法) :
3378 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3379 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
3380 // 2. FA 三板斧:
3381 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3382 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3383 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3384 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3385 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3386 //    - 优点: 减少 warp 间通信和 shuffle
3387 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3388 //    for each K,V tile:
3389 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3390 //        m_new = max(m_old, row_max(S_cur * scale))
3391 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3392 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3393 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3394 //        O_final = O_new / l_final
3395 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3396 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3397 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3398 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3399 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3400 //      都天然同构”的通用结论
3401 //

```

```

3402 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3403 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3404 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3405 // Block: (128, 1, 1), kNumThreads=WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
3406 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3407
3408 // ---- 寄存器填充辅助函数 ----
3409 template <typename T, int M, const int N, const int K = 2>
3410 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3411     #pragma unroll
3412     for (int i = 0; i < M; ++i)
3413     #pragma unroll
3414         for (int j = 0; j < N; ++j)
3415     #pragma unroll
3416         for (int k = 0; k < K; ++k)
3417             R[i][j][k] = val;
3418 }
3419
3420 template <typename T, int M, const int N = 2>
3421 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3422     #pragma unroll
3423     for (int i = 0; i < M; ++i)
3424     #pragma unroll
3425         for (int j = 0; j < N; ++j)
3426             R[i][j] = val;
3427 }
3428
3429 // =====
3430 // FlashAttention-2 Split-Q Kernel (完整实现)
3431 // =====
3432 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3433 //
3434 // Tile 设计 (以 kHeadDim=64 为例) :
3435 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3436 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3437 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3438 //
3439 // 执行流程:
3440 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3441 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3442 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3443 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
3444 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3445 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3446 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3447 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3448 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3449 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3450 //   9) Epilogue: warp shuffle + 128-bit collective store
3451
3452 template <
3453     const int kHeadDim,           // head dim: 32, 64, 128
3454     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3455     const int kMmaAtomN,          // 8  (MMA instruction N dimension)
3456     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3457     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3458     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3459     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3460     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3461     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3462     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3463     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3464     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)

```

```

3465     const int kStage,           // pipeline stages for K: 1 or 2
3466     const int kPad>           // padding for bank conflict avoidance
3467 __global__ void __launch_bounds__(WARP_SIZE *kMmaTileSeqLenQ *kMmaTileSeqLenK)
3468     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3469                                   int QKV_seqlen, int QKV_head) {
3470     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3471     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3472     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3473     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3474     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3475     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3476     const float scale = 1.0f / sqrtf((float)kHeadDim);
3477
3478     // Block indexing
3479     const int QKV_batch_id = blockIdx.y / QKV_head;
3480     const int QKV_head_id = blockIdx.y % QKV_head;
3481     const int Q_tile_id = blockIdx.x;
3482     const int tid = threadIdx.x;
3483     const int warp_id = tid / WARP_SIZE;
3484     const int lane_id = tid % WARP_SIZE;
3485     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3486     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3487
3488     // Global memory base offsets for this (batch, head)
3489     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3490     const int Q_gmem_offset =
3491         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3492         kHeadDim;
3493     const int K_gmem_offset = Q_gmem_offset;
3494     const int V_gmem_offset = Q_gmem_offset;
3495     const int O_gmem_offset = Q_gmem_offset;
3496
3497     // Thread-to-smem mapping for cooperative load
3498     int load_smem_Q_Br = tid / (kNumThreads / Br);
3499     int load_smem_Q_d =
3500         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3501     int load_smem_K_Bc = tid / (kNumThreads / Bc);
3502     int load_smem_K_d =
3503         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3504     int load_smem_V_Bc = tid / (kNumThreads / Bc);
3505     int load_smem_V_d =
3506         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3507
3508     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3509     if (load_gmem_Q_Br >= QKV_seqlen)
3510         return;
3511
3512     // ---- Shared memory layout ----
3513     extern __shared__ half smem[];
3514     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3515     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3516     half *Q_tile_smem = smem;
3517     half *K_tile_smem = Q_tile_smem + Q_tile_size;
3518     half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3519     // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3520     // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3521
3522     uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3523     uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3524     uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3525
3526     // ---- Online Softmax persistent state ----
3527     // lane_block_row_max_old[i][r]: running max for row r of warp tile i

```



```

3528 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3529 float lane_block_row_max_old[kValTileSeqLenQ][2];
3530 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3531 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3532 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3533
3534 // ---- Register allocation ----
3535 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3536 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3537 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3538 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3539 // 对单个 m16n8k16 tile 而言:
3540 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
3541 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
3542 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3543 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3544 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3545 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3546           [2]; // O accumulator (final output)
3547
3548 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3549 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3550
3551 // =====
3552 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3553 // =====
3554 {
3555     int load_gmem_Q_addr =
3556         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3557     uint32_t load_smem_Q_ptr =
3558         smem_Q_base_ptr +
3559         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3560 #pragma unroll
3561     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3562         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3563     }
3564     CP_ASYNC_COMMIT_GROUP();
3565 }
3566
3567 // =====
3568 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3569 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
3570 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
3571 // =====
3572 if constexpr (kStage > 1) {
3573 #pragma unroll
3574     for (int stage = 0; stage < (kStage - 1); ++stage) {
3575         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3576         int load_gmem_K_addr =
3577             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3578         uint32_t load_smem_K_ptr =
3579             smem_K_base_ptr +
3580             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3581              load_smem_K_d) *
3582             sizeof(half);
3583 #pragma unroll
3584         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3585             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3586         }
3587         CP_ASYNC_COMMIT_GROUP();
3588     }
3589     CP_ASYNC_WAIT_GROUP(kStage - 2);
3590     __syncthreads();

```

```

3591 }
3592
3593 // =====
3594 // Step 3: 外循环 — 沿 K seqLen 迭代 (Tc = ceil(seqLen/Bc))
3595 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3596 // =====
3597 #pragma unroll 1
3598 for (int tile_K_seqLen = 0; tile_K_seqLen < Tc; ++tile_K_seqLen) {
3599     int smem_sel = tile_K_seqLen % kStage;
3600     int smem_sel_next = (tile_K_seqLen + (kStage - 1)) % kStage;
3601
3602     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3603     if constexpr (kStage > 1) {
3604         // 只有 kStage>1 才能真正做 K 的 pipeline:
3605         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
3606         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3607         // Load current V tile (no pipeline for V — one stage is enough)
3608         {
3609             int load_gmem_V_Bc = tile_K_seqLen * Bc + load_smem_V_Bc;
3610             int load_gmem_V_addr =
3611                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3612             uint32_t load_smem_V_ptr =
3613                 smem_V_base_ptr +
3614                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3615 #pragma unroll
3616 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3617     CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3618 }
3619 CP_ASYNC_COMMIT_GROUP();
3620 }
3621
3622 // Prefetch next K tile (pipelined)
3623 if ((tile_K_seqLen + 1) < Tc) {
3624     int load_gmem_K_Bc = (tile_K_seqLen + 1) * Bc + load_smem_K_Bc;
3625     int load_gmem_K_addr =
3626         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3627     uint32_t load_smem_K_ptr =
3628         smem_K_base_ptr +
3629         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3630          load_smem_K_d) *
3631         sizeof(half);
3632 #pragma unroll
3633 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3634     CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3635 }
3636 CP_ASYNC_COMMIT_GROUP();
3637 }
3638 }
3639
3640 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3641 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3642 #pragma unroll
3643 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3644     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3645 #pragma unroll
3646 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3647     int warp_smem_Q_Br =
3648         warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3649     int lane_smem_Q_Br =
3650         warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3651     int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3652     uint32_t lane_smem_Q_ptr =
3653         smem_Q_base_ptr +

```

```

3654         (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3655     LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3656         lane_smem_Q_ptr);
3657 }
3658
3659 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3660 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3661 #pragma unroll
3662 for (int j = 0; j < kValTileSeqLenK; ++j) {
3663     int warp_smem_K_Bc =
3664         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3665     int lane_smem_K_Bc =
3666         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3667     int lane_smem_K_d =
3668         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3669     uint32_t lane_smem_K_ptr =
3670         smem_K_base_ptr +
3671         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3672         lane_smem_K_d) *
3673         sizeof(half);
3674     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3675 }
3676
3677 // MMA: S[tile] += Q[tile] @ K^T[tile]
3678 #pragma unroll
3679 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3680     #pragma unroll
3681     for (int j = 0; j < kValTileSeqLenK; ++j) {
3682         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3683             R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3684             R_S[i][j][1]);
3685     }
3686 }
3687 } // end loop over d
3688 __syncthreads();
3689
3690 // =====
3691 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3692 // =====
3693 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3694 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3695 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3696 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3697 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3698 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3699 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3700 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3701 // kValTileSeqLenK tiles.
3702
3703 float lane_row_max_new[kValTileSeqLenQ][2];
3704 float lane_row_sum_new[kValTileSeqLenQ][2];
3705 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3706 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3707
3708 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3709 #pragma unroll
3710 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3711     #pragma unroll
3712     for (int j = 0; j < kValTileSeqLenK; ++j) {
3713         // Extract half values from R_S registers (C matrix fragment layout)
3714         float2 t_reg_S_0 =
3715             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3716         float2 t_reg_S_1 =

```

```

3717     __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3718     // S = (Q@K^T) * scale
3719     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3720     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3721     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3722     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3723 }
3724 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3725 // {0,4,8,...,28} hold valid data)
3726 lane_row_max_new[i][0] =
3727     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3728 lane_row_max_new[i][1] =
3729     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3730 }
3731
3732 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3733 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3734 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3735 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3736 #pragma unroll
3737 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3738     // m_new = max(m_old, m_cur)
3739     float block_row_max_new_0 =
3740         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3741     float block_row_max_new_1 =
3742         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3743
3744     #pragma unroll
3745     for (int j = 0; j < kValTileSeqLenK; ++j) {
3746         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3747         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3748
3749         // P = exp(S * scale - m_new), 用 fma 保证精度
3750         t_reg_S_0.x =
3751             __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3752         t_reg_S_0.y =
3753             __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3754         t_reg_S_1.x =
3755             __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3756         t_reg_S_1.y =
3757             __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3758
3759         // Accumulate row sums
3760         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3761         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3762
3763         // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3764         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3765         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3766     }
3767
3768     // Warp-level reduce sum (warp_size = 4, same as max)
3769     lane_row_sum_new[i][0] =
3770         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3771     lane_row_sum_new[i][1] =
3772         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3773 }
3774 __syncthreads();
3775
3776 // =====
3777 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
3778 // =====
3779 // Wait for V to be ready before computing P@V

```

```

3780 if constexpr (kStage > 1) {
3781     if ((tile_K_seqLen + 1) < Tc) {
3782         CP_ASYNC_WAIT_GROUP(1);
3783     } else {
3784         CP_ASYNC_WAIT_GROUP(0);
3785     }
3786 } else {
3787     CP_ASYNC_WAIT_GROUP(0);
3788 }
3789 __syncthreads();
3790
3791 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3792
3793 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3794 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3795 #pragma unroll
3796 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3797     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3798     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3799     // transposed ldmatrix
3800 #pragma unroll
3801 for (int j = 0; j < kValTileHeadDimV; ++j) {
3802     int warp_smem_V_d =
3803         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3804     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3805     int lane_smem_V_d = warp_smem_V_d;
3806     uint32_t lane_smem_V_ptr =
3807         smem_V_base_ptr +
3808         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3809     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3810 }
3811
3812 // P matrix layout in R_S[i][j][2]:
3813 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3814 //   | 64x64 | warp_KV 0 |
3815 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3816 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3817 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3818 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3819 // tile_V_Bc selects which 16-column slice of P to use:
3820 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3821 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3822 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3823 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3824 // 对应的 MMA A fragment 布局可以这样记:
3825 //   - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
3826 //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
3827 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3828 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3829 // fragment 都无条件成立的通用结论。layout转换逻辑:
3830 //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
3831 int w = tile_V_Bc * 2;
3832 #pragma unroll
3833 for (int i = 0; i < kValTileSeqLenP; ++i) {
3834 #pragma unroll
3835 for (int j = 0; j < kValTileHeadDimV; ++j) {
3836     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
3837         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3838         R_V[j][0], R_V[j][1], // B fragment = V
3839         R_0[i][j][0], R_0[i][j][1]); // C fragment output
3840     }
3841 }
3842 } // end for tile_V_Bc

```

```

3843 __syncthreads();
3844
3845 // =====
3846 // 3e: Online rescaling —  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$ 
3847 // =====
3848 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{old} - m_{new})$  做  $O$  与  $l$  的 rescale
3849 #pragma unroll
3850 for (int i = 0; i < kValTileSeqLenP; ++i) {
3851     float block_row_max_new_0 = lane_row_max_new[i][0];
3852     float block_row_max_new_1 = lane_row_max_new[i][1];
3853     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3854     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3855
3856     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3857     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3858
3859     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3860     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3861
3862     // Handle first iteration:  $m_{old} = -inf$ , need to use  $m_{new}$  directly
3863     block_row_max_old_0 =
3864         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3865     block_row_max_old_1 =
3866         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3867
3868     float rescale_o_factor_0 =
3869         __expf(block_row_max_old_0 - block_row_max_new_0);
3870     float rescale_o_factor_1 =
3871         __expf(block_row_max_old_1 - block_row_max_new_1);
3872
3873     // Rescale  $O_{old}$  + Add  $P@V$  in one fused step
3874 #pragma unroll
3875     for (int j = 0; j < kValTileHeadDimV; ++j) {
3876         //  $R_0 / R_D$  与前面的  $R_S$  一样, 都按 MMA C fragment 布局解释:
3877         // reg[0] -> rows 0~7 的 {c0,c1}
3878         // reg[1] -> rows 8~15 的 {c2,c3}
3879         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3880         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3881         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3882         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3883
3884         //  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$  (fused multiply-add)
3885         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3886         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3887         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3888         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3889
3890         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3891         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3892     }
3893
3894     // Update  $l$ :  $l_{new} = \exp(m_{old} - m_{new}) * l_{old} + row\_sum(P)$ 
3895     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3896     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3897     lane_block_row_sum_old[i][0] = __fmaf_rn(
3898         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3899     lane_block_row_sum_old[i][1] = __fmaf_rn(
3900         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3901
3902     // Update  $m$ 
3903     lane_block_row_max_old[i][0] = block_row_max_new_0;
3904     lane_block_row_max_old[i][1] = block_row_max_new_1;
3905 }

```



```

3906 // Wait for next K tile to be ready in smem before next iteration
3907 if constexpr (kStage > 1) {
3908     if ((tile_K_seqlen + 1) < Tc) {
3909         CP_ASYNC_WAIT_GROUP(0);
3910     }
3911     __syncthreads();
3912 }
3913 } // end outer loop over K seqlen
3914 __syncthreads();
3915
3916 // =====
3917 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
3918 // =====
3919
3920 #pragma unroll
3921 for (int i = 0; i < kValTileSeqLenP; ++i) {
3922     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3923     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
3924     #pragma unroll
3925     for (int j = 0; j < kValTileHeadDimV; ++j) {
3926         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3927         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3928         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3929         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3930         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3931         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3932         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3933         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3934     }
3935 }
3936
3937 // =====
3938 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3939 // =====
3940 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3941 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3942 #pragma unroll
3943 for (int i = 0; i < kValTileSeqLenP; ++i) {
3944     #pragma unroll
3945     for (int j = 0; j < kValTileHeadDimV; ++j) {
3946         uint32_t R_Z[2][4];
3947         R_Z[0][0] = R_D[i][j][0];
3948         R_Z[1][0] = R_D[i][j][1];
3949         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3950         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3951         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3952         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3953         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3954         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3955
3956         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3957         if (lane_id % 4 == 0) {
3958             int store_warp_regs_0_Br =
3959                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3960             int store_lane_gmem_0_Br =
3961                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3962             int store_warp_regs_0_d =
3963                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3964             int store_lane_gmem_0_d = store_warp_regs_0_d;
3965             int store_gmem_0_addr_0 = 0_gmem_offset +
3966                 (store_lane_gmem_0_Br + 0) * kHeadDim +
3967                 store_lane_gmem_0_d;
3968             int store_gmem_0_addr_1 = 0_gmem_offset +

```

```

3969         (store_lane_gmem_0_Br + 8) * kHeadDim +
3970         store_lane_gmem_0_d;
3971     // LDST128BITS = reinterpret_cast<float4*>
3972     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
3973         *reinterpret_cast<float4*>(&R_Z[0][0]);
3974     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
3975         *reinterpret_cast<float4*>(&R_Z[1][0]);
3976 }
3977 }
3978 }
3979 }
3980
3981 // =====
3982 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
3983 // =====
3984
3985 static inline void check(cudaError_t err, const char *msg) {
3986     if (err != cudaSuccess) {
3987         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
3988         exit(EXIT_FAILURE);
3989     }
3990 }
3991
3992
3993 static void test_block_reduce(int N) {
3994
3995     srand(42);
3996     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3997     for (int i = 0; i < N; i++)
3998         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3999
4000     // CPU reference: sum of all elements
4001     double ref = 0.0;
4002     for (int i = 0; i < N; i++) ref += (double)h_a[i];
4003
4004     float *d_a, *d_y;
4005     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4006     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4007
4008     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
4009     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4010
4011     dim3 block(128);
4012     dim3 grid((N + 127) / 128);
4013     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4014     check(cudaGetLastError(), "blockreduce launch");
4015     check(cudaDeviceSynchronize(), "blockreduce sync");
4016
4017     float result;
4018     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4019
4020     float err = fabsf(result - (float)ref);
4021     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
4022         err < 1e-2f ? "PASS" : "FAIL");
4023
4024     free(h_a);
4025     cudaFree(d_a); cudaFree(d_y);
4026 }
4027
4028
4029 static void test_dot(int N) {
4030
4031     srand(42);

```

```

4032 float *h_a = (float *)malloc((size_t)N * sizeof(float));
4033 float *h_b = (float *)malloc((size_t)N * sizeof(float));
4034 for (int i = 0; i < N; i++) {
4035     h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4036     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4037 }
4038
4039 // CPU reference
4040 double ref = 0.0;
4041 for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4042
4043 float *d_a, *d_b, *d_y;
4044 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4045 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4046 check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4047
4048 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4049 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4050 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4051
4052 dim3 block(128);
4053 dim3 grid((N + 127) / 128);
4054 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4055 check(cudaGetLastError(), "dot launch");
4056 check(cudaDeviceSynchronize(), "dot sync");
4057
4058 float result;
4059 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4060
4061 float err = fabsf(result - (float)ref);
4062 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
4063     err < 1e-2f ? "PASS" : "FAIL");
4064
4065 // ---- Dot Vec4 ----
4066 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4067 dim3 block_v4(32);
4068 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4069 check(cudaGetLastError(), "dot_vec4 launch");
4070 check(cudaDeviceSynchronize(), "dot_vec4 sync");
4071
4072 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4073 float err_v4 = fabsf(result - (float)ref);
4074 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
4075
4076 free(h_a); free(h_b);
4077 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4078 }
4079
4080
4081 static void test_relu(int N) {
4082     srand(42);
4083     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4084     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4085     for (int i = 0; i < N; i++)
4086         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4087
4088     float *d_x, *d_y;
4089     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4090     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4091     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4092
4093     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4094     dim3 block256(256);

```

```

4095 dim3 grid256((N + 255) / 256);
4096 relu<<<grid256, block256>>>(d_x, d_y, N);
4097 check(cudaGetLastError(), "relu launch");
4098 check(cudaDeviceSynchronize(), "relu sync");
4099 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4100 float max_err = 0.0f;
4101 for (int i = 0; i < N; i++) {
4102     float expected = fmaxf(0.0f, h_x[i]);
4103     float err = fabsf(h_y[i] - expected);
4104     if (err > max_err) max_err = err;
4105 }
4106 printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4107
4108 dim3 block64(64);
4109 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4110 check(cudaGetLastError(), "relu_vec4 launch");
4111 check(cudaDeviceSynchronize(), "relu_vec4 sync");
4112 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4113 max_err = 0.0f;
4114 for (int i = 0; i < N; i++) {
4115     float expected = fmaxf(0.0f, h_x[i]);
4116     float err = fabsf(h_y[i] - expected);
4117     if (err > max_err) max_err = err;
4118 }
4119 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4120
4121 free(h_x); free(h_y);
4122 cudaFree(d_x); cudaFree(d_y);
4123 }
4124
4125
4126 static void test_elementwise(int N) {
4127     srand(42);
4128     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4129     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4130     for (int i = 0; i < N; i++) {
4131         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4132         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4133     }
4134
4135     float *d_a, *d_b, *d_c;
4136     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4137     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4138     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4139     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4140     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4141
4142     dim3 block256(256);
4143     dim3 grid256((N + 255) / 256);
4144     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4145     check(cudaGetLastError(), "eadd launch");
4146     check(cudaDeviceSynchronize(), "eadd sync");
4147     float *h_c = (float *)malloc((size_t)N * sizeof(float));
4148     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4149     float max_err = 0.0f;
4150     for (int i = 0; i < N; i++) {
4151         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4152         if (err > max_err) max_err = err;
4153     }
4154     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4155
4156     dim3 block64(64);
4157     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");

```

```

4158 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4159 check(cudaGetLastError(), "eadd_vec4 launch");
4160 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4161 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4162 max_err = 0.0f;
4163 for (int i = 0; i < N; i++) {
4164     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4165     if (err > max_err) max_err = err;
4166 }
4167 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4168
4169 free(h_a); free(h_b); free(h_c);
4170 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4171 }
4172
4173
4174 static void test_histogram(int N) {
4175     srand(42);
4176     int BINS = 16;
4177     int *h_hist = (int *)calloc(BINS, sizeof(int));
4178     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4179     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4180     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4181     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4182
4183     int *d_idx, *d_hist;
4184     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4185     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4186     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4187     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4188
4189     dim3 block256(256);
4190     dim3 grid256((N + 255) / 256);
4191     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4192     check(cudaGetLastError(), "histogram launch");
4193     check(cudaDeviceSynchronize(), "histogram sync");
4194     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4195
4196     float max_err = 0.0f;
4197     for (int i = 0; i < BINS; i++) {
4198         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4199         if (err > max_err) max_err = err;
4200     }
4201     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4202
4203     free(h_hist); free(h_hist_ref); free(h_idx);
4204     cudaFree(d_idx); cudaFree(d_hist);
4205 }
4206
4207
4208 static void test_merge_attn_states(int num_tokens, int num_heads,
4209                                   int head_size) {
4210
4211     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4212     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4213
4214     float *h_prefix_out = (float *)malloc(out_size);
4215     float *h_suffix_out = (float *)malloc(out_size);
4216     float *h_prefix_lse = (float *)malloc(lse_size);
4217     float *h_suffix_lse = (float *)malloc(lse_size);
4218     float *h_output_ref = (float *)malloc(out_size);
4219
4220     srand(42);

```

```

4221 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4222     h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4223     h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4224 }
4225 for (int i = 0; i < num_heads * num_tokens; i++) {
4226     h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4227     h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4228 }
4229
4230 // CPU reference: 与 kernel 完全相同的浮点计算
4231 for (int t = 0; t < num_tokens; t++) {
4232     for (int h = 0; h < num_heads; h++) {
4233         float p_lse = h_prefix_lse[h * num_tokens + t];
4234         float s_lse = h_suffix_lse[h * num_tokens + t];
4235         p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4236         s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4237
4238         float max_lse = fmaxf(p_lse, s_lse);
4239         p_lse -= max_lse;
4240         s_lse -= max_lse;
4241         float p_se = expf(p_lse);
4242         float s_se = expf(s_lse);
4243         float p_scale = p_se / (p_se + s_se);
4244         float s_scale = s_se / (p_se + s_se);
4245
4246         int head_off = t * num_heads * head_size + h * head_size;
4247         for (int d = 0; d < head_size; d++) {
4248             h_output_ref[head_off + d] =
4249                 h_prefix_out[head_off + d] * p_scale +
4250                 h_suffix_out[head_off + d] * s_scale;
4251         }
4252     }
4253 }
4254
4255 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4256 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4257 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4258 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4259 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4260 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4261
4262 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4263                 cudaMemcpyHostToDevice),
4264        "merge_attn H2D p_out");
4265 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4266                 cudaMemcpyHostToDevice),
4267        "merge_attn H2D s_out");
4268 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4269                 cudaMemcpyHostToDevice),
4270        "merge_attn H2D p_lse");
4271 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4272                 cudaMemcpyHostToDevice),
4273        "merge_attn H2D s_lse");
4274
4275 int threads_per_head = head_size / 4;
4276 int total_threads = num_tokens * num_heads * threads_per_head;
4277 dim3 block(128);
4278 dim3 grid((total_threads + 127) / 128);
4279 merge_attn_states<<<grid, block>>>(
4280     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4281     num_tokens, num_heads, head_size);
4282 check(cudaGetLastError(), "merge_attn launch");
4283 check(cudaDeviceSynchronize(), "merge_attn sync");

```



```

4284
4285 float *h_output = (float *)malloc(out_size);
4286 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4287        "merge_attn D2H");
4288
4289 float max_err = 0.0f;
4290 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4291     float err = fabsf(h_output[i] - h_output_ref[i]);
4292     if (err > max_err) max_err = err;
4293 }
4294 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
4295        max_err < 1e-4f ? "PASS" : "FAIL");
4296
4297 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4298 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4299 h_suffix_lse[0] = 0.0f;
4300 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4301                 cudaMemcpyHostToDevice),
4302        "merge_attn H2D inf lse");
4303 merge_attn_states<<<grid, block>>>(
4304     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4305     num_tokens, num_heads, head_size);
4306 check(cudaGetLastError(), "merge_attn inf launch");
4307 check(cudaDeviceSynchronize(), "merge_attn inf sync");
4308 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4309        "merge_attn inf D2H");
4310
4311 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4312 float inf_err = 0.0f;
4313 for (int d = 0; d < head_size; d++) {
4314     float err = fabsf(h_output[d] - h_suffix_out[d]);
4315     if (err > inf_err) inf_err = err;
4316 }
4317 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
4318        inf_err < 1e-4f ? "PASS" : "FAIL");
4319
4320 free(h_prefix_out);
4321 free(h_suffix_out);
4322 free(h_prefix_lse);
4323 free(h_suffix_lse);
4324 free(h_output_ref);
4325 free(h_output);
4326 cudaFree(d_prefix_out);
4327 cudaFree(d_suffix_out);
4328 cudaFree(d_prefix_lse);
4329 cudaFree(d_suffix_lse);
4330 cudaFree(d_output);
4331 }
4332
4333
4334 static void test_softmax(int N) {
4335     // Use N = blockDim.x so one block processes all elements as one token.
4336     constexpr int NUM_THREADS = 256;
4337     if (N != NUM_THREADS) {
4338         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
4339         return;
4340     }
4341
4342     srand(42);
4343     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4344     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4345     for (int i = 0; i < N; i++)
4346         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]

```

```

4347 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4348 float max_val = -FLT_MAX;
4349 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4350 double sum_exp = 0.0;
4351 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4352 for (int i = 0; i < N; i++)
4353     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4354
4355 float *d_x, *d_y;
4356 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4357 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4358
4359 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4360
4361 dim3 block(256);
4362 dim3 grid(1); // one token covering all N elements
4363 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4364 check(cudaGetLastError(), "softmax launch");
4365 check(cudaDeviceSynchronize(), "softmax sync");
4366
4367 float *h_y = (float *)malloc((size_t)N * sizeof(float));
4368 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4369
4370 float max_err = 0.0f;
4371 for (int i = 0; i < N; i++) {
4372     float err = fabsf(h_y[i] - h_y_ref[i]);
4373     if (err > max_err) max_err = err;
4374 }
4375 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4376
4377 // ---- Safe Softmax ----
4378 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4379 check(cudaGetLastError(), "safe_softmax launch");
4380 check(cudaDeviceSynchronize(), "safe_softmax sync");
4381 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4382 max_err = 0.0f;
4383 for (int i = 0; i < N; i++) {
4384     float err = fabsf(h_y[i] - h_y_ref[i]);
4385     if (err > max_err) max_err = err;
4386 }
4387 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4388
4389 // ---- Naive Softmax ----
4390 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4391 check(cudaGetLastError(), "naive_softmax launch");
4392 check(cudaDeviceSynchronize(), "naive_softmax sync");
4393 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4394 max_err = 0.0f;
4395 for (int i = 0; i < N; i++) {
4396     float err = fabsf(h_y[i] - h_y_ref[i]);
4397     if (err > max_err) max_err = err;
4398 }
4399 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4400
4401 free(h_x); free(h_y); free(h_y_ref);
4402 cudaFree(d_x); cudaFree(d_y);
4403 }
4404
4405 static void test_rms_norm(int N, int K) {
4406
4407     srand(42);

```

```

4410 float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4411 float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4412 for (int i = 0; i < N * K; i++)
4413     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4414 float g = 1.5f; // gain
4415
4416 // CPU reference: y = (x / rms(x)) * g
4417 float epsilon = 1e-5f;
4418 for (int n = 0; n < N; n++) {
4419     double sum_sq = 0.0;
4420     for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4421     float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4422     for (int k = 0; k < K; k++)
4423         h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4424 }
4425
4426 float *d_x, *d_y;
4427 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4428 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4429 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4430
4431 dim3 block(128);
4432 dim3 grid(N);
4433 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4434 check(cudaGetLastError(), "rmsnorm launch");
4435 check(cudaDeviceSynchronize(), "rmsnorm sync");
4436
4437 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4438 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4439
4440 float max_err = 0.0f;
4441 for (int i = 0; i < N * K; i++) {
4442     float err = fabsf(h_y[i] - h_y_ref[i]);
4443     if (err > max_err) max_err = err;
4444 }
4445 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4446
4447 // ---- RMS Norm Vec4 ----
4448 dim3 block_rv4(32);
4449 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4450 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4451 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4452 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4453 max_err = 0.0f;
4454 for (int i = 0; i < N * K; i++) {
4455     float err = fabsf(h_y[i] - h_y_ref[i]);
4456     if (err > max_err) max_err = err;
4457 }
4458 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4459
4460 free(h_x); free(h_y); free(h_y_ref);
4461 cudaFree(d_x); cudaFree(d_y);
4462 }
4463
4464
4465 static void test_layer_norm(int N, int K) {
4466
4467     srand(42);
4468     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4469     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4470     for (int i = 0; i < N * K; i++)
4471         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4472     float g = 1.5f, b = 0.3f; // gain and bias

```

```

4473 // CPU reference:  $y = ((x - \text{mean}) / \text{std}) * g + b$ 
4474 float epsilon = 1e-5f;
4475 for (int n = 0; n < N; n++) {
4476     double sum = 0.0;
4477     for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4478     float mean = (float)sum / (float)K;
4479     double sum_sq = 0.0;
4480     for (int k = 0; k < K; k++) {
4481         float diff = h_x[n * K + k] - mean;
4482         sum_sq += (double)diff * (double)diff;
4483     }
4484     float std = sqrtf((float)sum_sq / (float)K + epsilon);
4485     for (int k = 0; k < K; k++)
4486         h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4487 }
4488
4489 float *d_x, *d_y;
4490 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4491 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4492 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4493
4494 dim3 block(128);
4495 dim3 grid(N);
4496 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4497 check(cudaGetLastError(), "layernorm launch");
4498 check(cudaDeviceSynchronize(), "layernorm sync");
4499
4500 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4501 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4502
4503 float max_err = 0.0f;
4504 for (int i = 0; i < N * K; i++) {
4505     float err = fabsf(h_y[i] - h_y_ref[i]);
4506     if (err > max_err) max_err = err;
4507 }
4508 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4509
4510 // ---- Layer Norm Vec4 ----
4511 dim3 block_lv4(32);
4512 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4513 check(cudaGetLastError(), "layernorm_vec4 launch");
4514 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4515 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4516 max_err = 0.0f;
4517 for (int i = 0; i < N * K; i++) {
4518     float err = fabsf(h_y[i] - h_y_ref[i]);
4519     if (err > max_err) max_err = err;
4520 }
4521 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4522
4523 free(h_x); free(h_y); free(h_y_ref);
4524 cudaFree(d_x); cudaFree(d_y);
4525 }
4526
4527
4528 static void test_rope(int seq_len, int N) {
4529
4530     int total_pairs = seq_len * N;
4531     int total_elems = total_pairs * 2;
4532     size_t size = (size_t)total_elems * sizeof(float);
4533
4534     float *h_x = (float *)malloc(size);

```

```

4536 float *h_y_ref = (float *)malloc(size);
4537
4538 srand(42);
4539 for (int i = 0; i < total_elems; i++)
4540     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4541
4542 // CPU reference: 2D rotation for each pair
4543 for (int idx = 0; idx < total_pairs; idx++) {
4544     int token_pos = idx / N;
4545     int token_idx = idx % N;
4546     float x1 = h_x[idx * 2];
4547     float x2 = h_x[idx * 2 + 1];
4548     float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4549     float angle = (float)token_pos * theta;
4550     float cos_v = cosf(angle);
4551     float sin_v = sinf(angle);
4552     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4553     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4554 }
4555
4556 float *d_x, *d_y;
4557 check(cudaMalloc(&d_x, size), "rope alloc X");
4558 check(cudaMalloc(&d_y, size), "rope alloc Y");
4559 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4560
4561 dim3 block(256);
4562 dim3 grid((total_pairs + 255) / 256);
4563 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4564 check(cudaGetLastError(), "rope launch");
4565 check(cudaDeviceSynchronize(), "rope sync");
4566
4567 float *h_y = (float *)malloc(size);
4568 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4569
4570 float max_err = 0.0f;
4571 for (int i = 0; i < total_elems; i++) {
4572     float err = fabsf(h_y[i] - h_y_ref[i]);
4573     if (err > max_err) max_err = err;
4574 }
4575 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4576
4577 free(h_x);
4578 free(h_y);
4579 free(h_y_ref);
4580 cudaFree(d_x);
4581 cudaFree(d_y);
4582 }
4583
4584
4585 static void test_mat_transpose(int row, int col) {
4586
4587     size_t size_in = (size_t)row * col * sizeof(float);
4588     size_t size_out = (size_t)col * row * sizeof(float);
4589
4590     float *h_x = (float *)malloc(size_in);
4591     float *h_y_ref = (float *)malloc(size_out);
4592
4593     srand(42);
4594     for (int i = 0; i < row * col; i++)
4595         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4596
4597 // CPU reference: y[j][i] = x[i][j] (row-major)
4598 for (int i = 0; i < row; i++)

```

```

4599     for (int j = 0; j < col; j++)
4600         h_y_ref[j * row + i] = h_x[i * col + j];
4601
4602     float *d_x, *d_y;
4603     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4604     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4605     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4606
4607     dim3 block(16, 16);
4608     dim3 grid((col + 15) / 16, (row + 15) / 16);
4609     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4610     check(cudaGetLastError(), "mattrans launch");
4611     check(cudaDeviceSynchronize(), "mattrans sync");
4612
4613     float *h_y = (float *)malloc(size_out);
4614     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4615
4616     float max_err = 0.0f;
4617     for (int i = 0; i < col * row; i++) {
4618         float err = fabsf(h_y[i] - h_y_ref[i]);
4619         if (err > max_err) max_err = err;
4620     }
4621     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4622
4623     free(h_x);
4624     free(h_y);
4625     free(h_y_ref);
4626     cudaFree(d_x);
4627     cudaFree(d_y);
4628 }
4629
4630
4631 static void test_mat_transpose_padded(int row, int col) {
4632
4633     size_t size_in = (size_t)row * col * sizeof(float);
4634     size_t size_out = (size_t)col * row * sizeof(float);
4635
4636     float *h_x = (float *)malloc(size_in);
4637     float *h_y_ref = (float *)malloc(size_out);
4638
4639     srand(42);
4640     for (int i = 0; i < row * col; i++)
4641         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4642
4643     // CPU reference: y[j][i] = x[i][j] (row-major)
4644     for (int i = 0; i < row; i++)
4645         for (int j = 0; j < col; j++)
4646             h_y_ref[j * row + i] = h_x[i * col + j];
4647
4648     float *d_x, *d_y;
4649     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4650     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4651     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4652
4653     dim3 block(16, 16);
4654     dim3 grid((col + 15) / 16, (row + 63) / 64);
4655     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4656     check(cudaGetLastError(), "mattrans_padded launch");
4657     check(cudaDeviceSynchronize(), "mattrans_padded sync");
4658
4659     float *h_y = (float *)malloc(size_out);
4660     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4661

```



```

4662 float max_err = 0.0f;
4663 for (int i = 0; i < col * row; i++) {
4664     float err = fabsf(h_y[i] - h_y_ref[i]);
4665     if (err > max_err) max_err = err;
4666 }
4667 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4668
4669 free(h_x);
4670 free(h_y);
4671 free(h_y_ref);
4672 cudaFree(d_x);
4673 cudaFree(d_y);
4674 }
4675
4676
4677 static void test_sgemv(int M, int K) {
4678
4679     srand(42);
4680     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4681     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4682     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4683     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4684     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4685
4686     // CPU reference: y[m] = sum_k A[m,k] * x[k]
4687     for (int m = 0; m < M; m++) {
4688         double sum = 0.0;
4689         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4690         h_y_ref[m] = (float)sum;
4691     }
4692
4693     float *d_a, *d_x, *d_y;
4694     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4695     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4696     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4697
4698     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4699     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4700
4701     dim3 block(32, 4);
4702     dim3 grid((M + 3) / 4);
4703     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4704     check(cudaGetLastError(), "sgemv launch");
4705     check(cudaDeviceSynchronize(), "sgemv sync");
4706
4707     float *h_y = (float *)malloc((size_t)M * sizeof(float));
4708     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4709
4710     float max_err = 0.0f;
4711     for (int m = 0; m < M; m++) {
4712         float err = fabsf(h_y[m] - h_y_ref[m]);
4713         if (err > max_err) max_err = err;
4714     }
4715     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4716
4717     // ---- SGEMV K32 ----
4718     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4719     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4720     check(cudaGetLastError(), "sgemv_k32 launch");
4721     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4722
4723     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4724

```

```

4725 max_err = 0.0f;
4726 for (int m = 0; m < M; m++) {
4727     float err = fabsf(h_y[m] - h_y_ref[m]);
4728     if (err > max_err) max_err = err;
4729 }
4730 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4731
4732 // ---- SGEMV K16 ----
4733 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4734 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4735
4736 int K16 = 16;
4737 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4738 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4739 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4740 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4741 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4742
4743 for (int m = 0; m < M; m++) {
4744     double sum = 0.0;
4745     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4746     h_y_ref[m] = (float)sum;
4747 }
4748
4749 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4750 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4751 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4752
4753 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4754 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4755
4756 dim3 grid_k16((M + 7) / 8);
4757 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4758 check(cudaGetLastError(), "sgemv_k16 launch");
4759 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4760
4761 h_y = (float *)malloc((size_t)M * sizeof(float));
4762 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4763
4764 max_err = 0.0f;
4765 for (int m = 0; m < M; m++) {
4766     float err = fabsf(h_y[m] - h_y_ref[m]);
4767     if (err > max_err) max_err = err;
4768 }
4769 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4770
4771 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4772 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4773 }
4774
4775
4776 static void test_sgemm(int M, int N, int K) {
4777
4778     size_t size_a = (size_t)M * K * sizeof(float);
4779     size_t size_b = (size_t)K * N * sizeof(float);
4780     size_t size_c = (size_t)M * N * sizeof(float);
4781
4782     float *h_a = (float *)malloc(size_a);
4783     float *h_b = (float *)malloc(size_b);
4784     float *h_c_ref = (float *)malloc(size_c);
4785
4786     srand(42);
4787     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

4788 for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4789
4790 float *d_a, *d_b, *d_c;
4791 check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4792 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4793 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4794
4795 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4796 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4797
4798 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4799 cublasHandle_t handle;
4800 cublasCreate(&handle);
4801 float alpha = 1.0f, beta = 0.0f;
4802 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4803             &alpha, d_b, N, d_a, K, &beta, d_c, N);
4804 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4805
4806 // Kernel
4807 cudaEvent_t start, stop;
4808 cudaEventCreate(&start);
4809 cudaEventCreate(&stop);
4810
4811 dim3 block(32, 32);
4812 dim3 grid((N + 31) / 32, (M + 31) / 32);
4813 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4814 check(cudaGetLastError(), "sgemm launch");
4815 check(cudaDeviceSynchronize(), "sgemm sync");
4816
4817 float *h_c = (float *)malloc(size_c);
4818 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4819
4820 // Verify
4821 float max_err = 0.0f;
4822 for (int i = 0; i < M * N; i++) {
4823     float err = fabsf(h_c[i] - h_c_ref[i]);
4824     if (err > max_err) max_err = err;
4825 }
4826 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4827
4828 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4829
4830 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4831 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4832 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4833 check(cudaGetLastError(), "sgemm_vec4 launch");
4834 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4835
4836 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4837
4838 max_err = 0.0f;
4839 for (int i = 0; i < M * N; i++) {
4840     float err = fabsf(h_c[i] - h_c_ref[i]);
4841     if (err > max_err) max_err = err;
4842 }
4843 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4844
4845 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4846 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4847 cublasDestroy(handle);
4848 cudaEventDestroy(start);
4849 cudaEventDestroy(stop);
4850 }

```

```

4851
4852
4853 static void test_hgemmm_mma(int M, int N, int K) {
4854
4855     size_t size_a = (size_t)M * K * sizeof(half);
4856     size_t size_b = (size_t)K * N * sizeof(half);
4857     size_t size_c = (size_t)M * N * sizeof(half);
4858
4859     half *h_a = (half *)malloc(size_a);
4860     half *h_b = (half *)malloc(size_b);
4861     half *h_c_ref = (half *)malloc(size_c);
4862
4863     srand(42);
4864     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4865     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4866
4867     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4868     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
4869     size_t size_b_t = (size_t)N * K * sizeof(half);
4870     half *h_b_t = (half *)malloc(size_b_t);
4871     for (int n = 0; n < N; n++)
4872         for (int k = 0; k < K; k++)
4873             h_b_t[n * K + k] = h_b[k * N + n];
4874
4875     half *d_a, *d_b, *d_b_t, *d_c;
4876     check(cudaMalloc(&d_a, size_a), "hgemmm alloc A");
4877     check(cudaMalloc(&d_b, size_b), "hgemmm alloc B (cuBLAS)");
4878     check(cudaMalloc(&d_b_t, size_b_t), "hgemmm alloc B_t (kernel)");
4879     check(cudaMalloc(&d_c, size_c), "hgemmm alloc C");
4880
4881     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemmm H2D A");
4882     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemmm H2D B (cuBLAS)");
4883     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemmm H2D B_t (kernel)");
4884
4885     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4886     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4887     cublasHandle_t handle;
4888     cublasCreate(&handle);
4889     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4890     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4891                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4892                 &beta_h, d_c, CUDA_R_16F, N,
4893                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4894     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemmm D2H ref");
4895
4896     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4897     cudaEvent_t start, stop;
4898     cudaEventCreate(&start);
4899     cudaEventCreate(&stop);
4900
4901     constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
4902     size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
4903     dim3 block(256);
4904     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4905     hgemmm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4906     check(cudaGetLastError(), "hgemmm launch");
4907     check(cudaDeviceSynchronize(), "hgemmm sync");
4908
4909     half *h_c = (half *)malloc(size_c);
4910     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemmm D2H");
4911
4912     // Verify
4913     float max_err = 0.0f;

```

```

4914     for (int i = 0; i < M * N; i++) {
4915         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4916         if (err > max_err) max_err = err;
4917     }
4918     printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4919
4920     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4921     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4922     cublasDestroy(handle);
4923     cudaEventDestroy(start);
4924     cudaEventDestroy(stop);
4925 }
4926
4927
4928 static void test_hgemm_swizzle(int M, int N, int K) {
4929     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4930     // + Register Double Buffering (VAL_TILE_K=2, BK=32)
4931     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4932     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
4933     // (VAL_TILE_K=2, K_STAGE=3, BK=32)
4934     // smem: K_STAGE × (BM×BK + BN×BK) halves
4935
4936     size_t size_a = (size_t)M * K * sizeof(half);
4937     size_t size_b = (size_t)K * N * sizeof(half);
4938     size_t size_c = (size_t)M * N * sizeof(half);
4939
4940     half *h_a = (half *)malloc(size_a);
4941     half *h_b = (half *)malloc(size_b);
4942     half *h_c_ref = (half *)malloc(size_c);
4943
4944     srand(42);
4945     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4946     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4947
4948     // Kernel expects BT [N×K] row-major layout (TN layout convention).
4949     size_t size_b_t = (size_t)N * K * sizeof(half);
4950     half *h_b_t = (half *)malloc(size_b_t);
4951     for (int n = 0; n < N; n++)
4952         for (int k = 0; k < K; k++)
4953             h_b_t[n * K + k] = h_b[k * N + n];
4954
4955     half *d_a, *d_b, *d_b_t, *d_c;
4956     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4957     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4958     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4959     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4960
4961     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4962     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4963     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4964
4965     // cuBLAS FP16 reference
4966     cublasHandle_t handle;
4967     cublasCreate(&handle);
4968     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4969     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4970                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4971                 &beta_h, d_c, CUDA_R_16F, N,
4972                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4973     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
4974
4975     // MMA swizzle kernel (default params: K_STAGE=3, VAL_TILE_K=2, BK=32)
4976     constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;

```

```

4977 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
4978 dim3 block(256);
4979 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4980 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4981 check(cudaGetLastError(), "hgemm_swizzle launch");
4982 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
4983
4984 half *h_c = (half *)malloc(size_c);
4985 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
4986
4987 // Verify
4988 float max_err = 0.0f;
4989 for (int i = 0; i < M * N; i++) {
4990     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4991     if (err > max_err) max_err = err;
4992 }
4993 printf("| %-35s | %.6e | %-4s |\n", "HGEMM Swizzle+Reg2x", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4994
4995 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4996 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4997 cublasDestroy(handle);
4998 }
4999
5000
5001 #if defined(NOTES_V2_ENABLE_CUTE)
5002 static void test_hgemm_cute(int M, int N, int K) {
5003     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
5004     // TN layout: C[MxN] = A[MxK] x B^T[NxK]
5005     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5006     // Tile: BM=128, BN=256, BK=32, 128 threads/block
5007
5008     size_t size_a = (size_t)M * K * sizeof(half);
5009     size_t size_b = (size_t)K * N * sizeof(half);
5010     size_t size_c = (size_t)M * N * sizeof(half);
5011
5012     half *h_a = (half *)malloc(size_a);
5013     half *h_b = (half *)malloc(size_b);
5014     half *h_c_ref = (half *)malloc(size_c);
5015
5016     srand(42);
5017     for (int i = 0; i < M * K; i++)
5018         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5019     for (int i = 0; i < K * N; i++)
5020         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5021
5022     // CuTe kernel expects B^T [NxK] row-major (TN layout).
5023     size_t size_b_t = (size_t)N * K * sizeof(half);
5024     half *h_b_t = (half *)malloc(size_b_t);
5025     for (int n = 0; n < N; n++)
5026         for (int k = 0; k < K; k++)
5027             h_b_t[n * K + k] = h_b[k * N + n];
5028
5029     half *d_a, *d_b, *d_b_t, *d_c;
5030     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5031     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5032     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5033     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5034
5035     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5036     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5037     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5038
5039     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)

```



```

5040 cublasHandle_t handle;
5041 cublasCreate(&handle);
5042 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5043 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5044             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5045             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5046 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5047
5048 // CuTe kernel (Stages=2, TN layout)
5049 launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
5050 check(cudaGetLastError(), "hgemm_cute launch");
5051 check(cudaDeviceSynchronize(), "hgemm_cute sync");
5052
5053 half *h_c = (half *)malloc(size_c);
5054 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5055
5056 // Verify
5057 float max_err = 0.0f;
5058 for (int i = 0; i < M * N; i++) {
5059     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5060     if (err > max_err) max_err = err;
5061 }
5062 printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5063
5064 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5065 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5066 cublasDestroy(handle);
5067 }
5068 #endif /* NOTES_V2_ENABLE_CUTE */
5069
5070
5071 #if defined(NOTES_V2_ENABLE_WGMMMA)
5072 static void test_hgemm_wgmma(int M, int N, int K) {
5073     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
5074     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5075     // Kernel: hgemm_wgmma_stages_tn with default template params
5076
5077     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
5078
5079     // M, K must be divisible by tile dims
5080     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5081         printf("| %-35s | %-12s | %-4s |\n",
5082             "HGEMM WGMMMA", "SKIP", "SKIP");
5083         return;
5084     }
5085
5086     size_t size_a = (size_t)M * K * sizeof(half);
5087     size_t size_b = (size_t)K * N * sizeof(half);
5088     size_t size_c = (size_t)M * N * sizeof(half);
5089
5090     half *h_a = (half *)malloc(size_a);
5091     half *h_b = (half *)malloc(size_b);
5092     half *h_c_ref = (half *)malloc(size_c);
5093
5094     srand(42);
5095     for (int i = 0; i < M * K; i++)
5096         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5097     for (int i = 0; i < K * N; i++)
5098         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5099
5100     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
5101     size_t size_b_t = (size_t)N * K * sizeof(half);
5102     half *h_b_t = (half *)malloc(size_b_t);

```

```

5103 for (int n = 0; n < N; n++)
5104     for (int k = 0; k < K; k++)
5105         h_b_t[n * K + k] = h_b[k * N + n];
5106
5107 half *d_a, *d_b, *d_b_t, *d_c;
5108 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5109 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5110 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5111 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5112
5113 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5114 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5115         "wgmma H2D B (cuBLAS)");
5116 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5117         "wgmma H2D B_t (kernel)");
5118
5119 // cuBLAS FP16 reference (row-major idiom, same as hgemv_mma)
5120 cublasHandle_t handle;
5121 cublasCreate(&handle);
5122 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5123 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5124             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5125             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5126 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5127         "wgmma D2H ref");
5128
5129 // Create TMA tensor maps for A and B^T
5130 // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5131 // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5132 CUTensorMap *tma_a =
5133     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5134 CUTensorMap *tma_b =
5135     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5136
5137 // Launch WGMMMA kernel
5138 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
5139 size_t smem_bytes =
5140     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5141 cudaFuncSetAttribute(
5142     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
5143     false>,
5144     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5145
5146 dim3 block(NUM_THREADS);
5147 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5148 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
5149     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5150 check(cudaGetLastError(), "wgmma launch");
5151 check(cudaDeviceSynchronize(), "wgmma sync");
5152
5153 half *h_c = (half *)malloc(size_c);
5154 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5155
5156 // Verify
5157 float max_err = 0.0f;
5158 for (int i = 0; i < M * N; i++) {
5159     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5160     if (err > max_err)
5161         max_err = err;
5162 }
5163 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
5164         max_err < 1.0f ? "PASS" : "FAIL");
5165

```

```

5166 free(h_a);
5167 free(h_b);
5168 free(h_b_t);
5169 free(h_c);
5170 free(h_c_ref);
5171 cudaFree(d_a);
5172 cudaFree(d_b);
5173 cudaFree(d_b_t);
5174 cudaFree(d_c);
5175 cudaFree(tma_a);
5176 cudaFree(tma_b);
5177 cublasDestroy(handle);
5178 }
5179 #endif /* NOTES_V2_ENABLE_WGMMA */
5180
5181
5182 static void test_flash_attn(int seqlen, int head_dim) {
5183     // FlashAttention-2 with split-Q, MMA m16n8k16
5184     int B = 1, H = 8;
5185
5186     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5187
5188     srand(42);
5189     half *h_q = (half *)malloc(sz);
5190     half *h_k = (half *)malloc(sz);
5191     half *h_v = (half *)malloc(sz);
5192     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5193         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5194         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5195         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5196     }
5197
5198     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
5199     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5200     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5201     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5202     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5203     int count = B * H * seqlen * head_dim;
5204     for (int i = 0; i < count; i++) {
5205         ref_q[i] = __half2float(h_q[i]);
5206         ref_k[i] = __half2float(h_k[i]);
5207         ref_v[i] = __half2float(h_v[i]);
5208     }
5209
5210     float scale = 1.0f / sqrtf((float)head_dim);
5211     for (int bi = 0; bi < B * H; bi++) {
5212         for (int qi = 0; qi < seqlen; qi++) {
5213             // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
5214             float smax = -INFINITY;
5215             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5216             for (int kj = 0; kj < seqlen; kj++) {
5217                 float s = 0.0f;
5218                 for (int d = 0; d < head_dim; d++)
5219                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5220                        ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5221                 S[kj] = s * scale;
5222                 if (S[kj] > smax) smax = S[kj];
5223             }
5224             // softmax
5225             double sum_exp = 0.0;
5226             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5227             float inv_sum = 1.0f / (float)sum_exp;
5228             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]

```

```

5229     for (int d = 0; d < head_dim; d++) {
5230         double o_acc = 0.0;
5231         for (int kj = 0; kj < seqlen; kj++)
5232             o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5233                 ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5234         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5235     }
5236     free(S);
5237 }
5238 }
5239
5240 half *d_q, *d_k, *d_v, *d_o;
5241 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5242 check(cudaMalloc(&d_k, sz), "fa alloc K");
5243 check(cudaMalloc(&d_v, sz), "fa alloc V");
5244 check(cudaMalloc(&d_o, sz), "fa alloc O");
5245 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5246 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5247 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5248
5249 // Template params for kHeadDim=64, kStage=2
5250 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
5251 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
5252 constexpr int kMmaTileSeqLenQ = 4;
5253 constexpr int kMmaTileSeqLenK = 1;
5254 constexpr int kMmaTileSeqLenP = 4;
5255 constexpr int kMmaTileHeadDimV = 1;
5256 constexpr int kValTileSeqLenQ = 1;
5257 constexpr int kValTileSeqLenK = 8;
5258 constexpr int kValTileSeqLenP = 1;
5259 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5260
5261 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5262 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5263 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
5264                     kStageV * Bc * (kHeadDim + kPadV) +
5265                     Bc * (kHeadDim + kPadV)) * sizeof(half);
5266
5267 dim3 block(128);
5268 dim3 grid((seqlen + Br - 1) / Br, B * H);
5269
5270 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
5271     kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
5272     kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
5273     kStageV, kPadV>
5274     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
5275 check(cudaGetLastError(), "fa launch");
5276 check(cudaDeviceSynchronize(), "fa sync");
5277
5278 half *h_o = (half *)malloc(sz);
5279 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5280
5281 float max_err = 0.0f;
5282 for (int i = 0; i < count; i++) {
5283     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5284     if (err > max_err) max_err = err;
5285 }
5286 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
5287
5288 free(h_q); free(h_k); free(h_v); free(h_o);
5289 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5290 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5291 }

```

```

5292
5293
5294 int main(int argc, char *argv[]) {
5295 #if defined(NOTES_V2_ENABLE_WGMMMA)
5296     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
5297 #endif
5298     int M = 1024, N = 1024, K = 1024;
5299     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
5300
5301     printf("=== notes-v2.cu verification harness ===\n");
5302     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
5303     printf("|-----|-----|-----|\n");
5304
5305     test_block_reduce(N);
5306     test_dot(N);
5307     test_relu(1024);
5308     test_elementwise(1024);
5309     test_histogram(1024);
5310     test_merge_attn_states(512, 16, 128);
5311     test_softmax(256);
5312     test_rms_norm(8, 128);
5313     test_layer_norm(8, 128);
5314     test_rope(8, 128);
5315     test_mat_transpose(256, 256);
5316     test_mat_transpose_padded(256, 256);
5317     test_sgemv(256, 128);
5318     test_sgemm(M, N, K);
5319     test_hgemm_mma(M, N, K);
5320     test_hgemm_swizzle(M, N, K);
5321     #if (defined(NOTES_V2_ENABLE_CUTE))
5322         test_hgemm_cute(M, N, K);
5323     #endif
5324     #if (defined(NOTES_V2_ENABLE_WGMMMA))
5325         test_hgemm_wgmma(M, N, K);
5326     #endif
5327     test_flash_attn(1024, 64);
5328
5329     printf("=== All tests done ===\n");
5330     return 0;
5331 }
5332
5333 // =====
5334 // Quick build & run reference
5335 // =====
5336 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
5337 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
5338 // -I ../../third-party/cutlass/include \
5339 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
5340 //
5341 // # sm_89 单独编译 + 运行:
5342 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
5343 //
5344 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
5345 // 项目内置 third-party/cutlass) :
5346 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
5347 // -I ../../third-party/cutlass/include \
5348 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
5349 //
5350 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
5351 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5352 // -DNOTES_V2_ENABLE_WGMMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
5353 //
5354 // # sm_90a + CuTe + WGMMMA:

```

```
5355 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \  
5356 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \  
5357 // -I ../../third-party/cutlass/include \  
5358 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
```